Instruction-Fuel Drift, Boundary-Crossing Overhead, and Memory Safety in WebAssembly Sandboxes, spanning One more Multi-Runtime Measurement Study

Devarsh Patel[1] and Ranjithkumar S[1*]

[1*]School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India.

*Corresponding author(s). E-mail(s): ranjithkumar.s@vit.ac.in
E-mail: devarsh.patel2024@vitstudent.ac.in

Abstract

Multi-tenant WebAssembly platforms bound tenant CPU with instruction fuel; we measure how closely fuel monitors the CPU time a tenant consumes, how that relation rests on the runtime and the build, and what the sandbox boundary costs, at least in practice. In light of this dynamic, across 4,800 single-operation linear-memory probes every outcome conformed, with an exact 95% lower bound of **0.995** on conformance for each configuration, at least empirically; a solid 95th-to-5th percentile ratio of **8.9** is sharing that most cells are closer together than the extremes are disclosing, in real-world scenarios. Along these dimensions, within this framework, under Wasmtime's Cranelift backend the cost of one fuel unit differs by a factor of **40.8**, across cells; and thus, the same kernel's fuel differs by factors from **0.36** to **6.5** between −O0 and −O2 builds, at least for now; truly, also, winch and Cranelift are disagreeing on fuel in 20 cells above numerical noise, to a significant degree. Under these conditions, sandbox recycling adds about 9 bytes of private anonymous memory per lifecycle, independently of the sandbox's dirty pages and of the allocation strategy; chiefly, also identical checksums are returned in every cell by all setups, in many respects. From this perspective, building on this gather, building on this finding, each cell is timed under twelve runtime configurations on an 8-core system-on-chip; and thus, also, get 166,600 timed replicates; and thus, the corpus has 50 kernels (30 PolyBench/C, 10 integer and cryptographic,

and 10 linear-memory kernels) at four frontend optimization levels, giving 200 cells, all things considered. From this viewpoint, a host call costs about 61 times a native copy-and-validate at 64-byte payloads and reaches parity at 4 MiB, and WASI Preview 2 writes cost less per byte than Preview 1 writes; and thus, calibrating cost per fuel unit with opcode-class counts, whether static or dynamically conducted, does not reduce dispersion below the uncalibrated fuel baseline, broadly speaking.

Keywords: WebAssembly, serverless computing, sandboxing, fuel metering, runtime benchmarking, WASI, measurement methodology

1 Introduction

Most markedly, untrusted code in WebAssembly sandboxes is run by multi-tenant platforms, because the sandbox is starting quickly and separates memory [1, 2], to one significant degree, in actual application; at least in practice; also thus, memory isolation is only part of the contract, which is why this host also is needing any bound on the CPU one tenant can (to some degree) consume, in this primary, at least in practice. Along all this dimensions, also, most WebAssembly runtimes supply one by metering instructions; taken together, the meter is called fuel; this runtime decrements fuel as the guest executes and traps when the budget runs out, broadly speaking.

Remarkably, guest operations are, in this setting, counted by fuel. Under all the conditions, more exactly, CPU time rests on this machine code the runtime emits for them; two compilers sit between the two, like the frontend that is normally devising the module, and the backend that lowers it to native code, in actual application. Both are, in essence, changing the work any fixed source computation is doing; more correctly, also this backend is changing what each operation costs, at least in belief; the further fixed fuel budget therefore is buying mixed CPU time on separate modules, optimization levels, and runtimes, all things considered. Within this framework, under all this conditions, along all the dimensions, across one benchmark suite [3], jangda et al. measured WebAssembly against native code; to be precise, the question here is narrower, spanning how closely does metered fuel approximate the time any tenant consumes, in the central; also, how much does that approximation hinge on this runtime and the build?

We keily contribute:

- **Fuel-to-time drift and backend dependence.** For 50 kernels at four optimization levels we quantify this cost of one fuel unit, across the cells, broadly speaking; more exactly, also we uncover that the Winch and Cranelift backends report different fuel for the same module (Section 5.1), in real-world scenarios.
- **Cross-runtime cost with metering overhead.** Beyond this, all 200 cells under the twelve setups, are timed spanning metered and unmetered builds

of three Wasmer backends and both the interpreter and ahead-of-time modes of WasmEdge, all things considered; identical checksums (Section 5.1) are returned by all arrangements, to one more significant degree, in real-world scenarios.

- **Trap conformance.** 4,800 single-operation linear-memory probes are run, under six memory configurations, so also all of them are conforming (Section 5.2) in this broader context, at least for now.
- **Host-boundary and WASI cost.** We fit cost models to another custom host-call boundary and to WASI Preview 1 and Preview 2 writes, from 12,964 host samples and 12,964 WASI invocations (Section 5.3), in many respects.
- **Recycling and a negative calibration result.** Sandbox recycling is creating any stable growth term that is routinely autonomous of sandbox dirty pages, so also, calibration with static and with dynamically conducted opcode-class counts does not lower dispersion, at least for now; and thus, also, the failure is reported with its proof (Section 5.4, Section 5.5), on the whole.

In fact, Section 6 is, in this setting, turning all all the findings into the guidance for operators who are mostly setting tenant budgets in fuel, on the whole.

Most considerably, Section 2 is reviewing prior work. Building on this finding, also, section 3 is stating the metrics and the breakdown employed in the assessment, at least empirically; more correctly, Section 4 is recapping the testbed and telemetry, Section 5 reports the five research questions, and Section 6 turns them into the guidance for operators, at least in belief, in actual application. In light of this dynamic, section 8 is concluding.

2 Related Work

Beyond this, WebAssembly foundations and runtime studies are being spanned from 2017 to 2020 by the 44 bibliography entries, recent runtime, security and WASI work from 2024 to 2026, to one significant degree, and also the statistical and scheme references from 1934 to 2009 that Section 3 applies, in actual application. Against this backdrop, from this outlook, against this backdrop, table 1 places the closest empirical studies side by side, and the five clusters below group the rest, on the whole. To be precise, the search protocol ran ten keyword searches restricted to 2024 onward and six saturation checks that are counting recent papers on any candidate gap, at least empirically; chiefly, one record was kept only if it was any second peer-reviewed article or conference paper from one established publisher or indexed continuings, all things considered. Against this backdrop, also, only if its topic was runtime performance, sandbox isolation, host interfaces, or measurement methodology, at least in practice; thirty-two were repeats of any second record already seen, which left 92 unique data, at least empirically; also, together 124 entries were returned by them, at least in belief, in the central, broadly speaking. Taken

together, dynamic, theses, preprints, repo copies, and papers from venues outside the indexed databases were excluded; that left 28 entries, 19 of which were already in the validated reference list from one more further earlier pass; after their Crossref metadata matched their titles, the other nine were added, broadly speaking. Under these conditions, dynamic, building on this gather, the decision for every person person person person person record is logged in the store; the search index covers paywalled, in many respects; indeed, also open-access work alike, but its ranking favours recent and highly cited papers, so the search can miss one applicable paper that is neither, in the principal, in real-world scenarios.

The cluster descriptions state each work's scope and reported findings; and thus, each bibliography entry was inspected against its Crossref or USENIX record for title, year and venue, with a center on pages, in real-world scenarios.

The works are, in effect, grouped into the five clusters; in fact, also in parallel with this, the grouping is systematically ours, on the whole.

Cluster A: runtimes and cross-platform evaluation. Zhang et al. survey 103 research papers on WebAssembly runtimes, covering runtime design, testing and application spaces [4], in real-world scenarios. In parallel with this, moron and Wallentowitz benchmark WebAssembly on microcontrollers against native code, MicroPython and Lua [5], at least empirically. Khan et al. evaluate 13 WASI-compatible runtimes on eight machine-learning and arithmetic benchmarks, with the external power meter, and report that interpreter-oriented runtimes exceed 10× overhead in time and energy while optimized AOT engines approach one native reference [6], at least in practice. Kakati and Brorsson evaluate two runtimes, across the four architectures, spanning RISC-V and embedded boards, for speed and startup time [7], at least in law. Also, the performance and usability of multiplatform WebAssembly containers [8] are tackled by any later conference paper. Within this framework, building on this gather, de Palma et al. benchmark any WebAssembly function-as-a-service platform against three container-based platforms on restricted edge hardware [9]. Building on this note, from this outlook, khelifa et al. are finding roughly 1.1× overhead for small deep-learning networks on standalone runtimes, with larger networks sharing greater overhead [10]. Khan et al. compare machine-learning programs written in Python, Rust and C++ and gathered to WebAssembly, in one browser and, under WASI [11]. Golec et al. survey cold-start latency in serverless computing and advance one more taxonomy of mitigations [12] in this broader context. Against this backdrop, moebius et al. compare unikernels with Firecracker microVMs, Docker and gVisor for cold start and idle size at the edge [13]. Chennubhotla et al. report that WebAssembly configurations run 31.4% faster, use 59.5% less memory, and create images 86.5% smaller than Docker [14].

Cluster B: security, isolation, and resource control. Perrone and Romano are reviewing 121 papers on WebAssembly security, organized into the seven categories [15], at least for now; truly, rostamipoor et al. protect serverless functions against memory-leakage attacks with selective encryption

and report overheads of 2.8% to 8.5% [16], at least in practice. In this broader context, yu et al. decouple the WASI system interface into a virtualized daemon, which costs a 3.2× slowdown on the WASI call path but gives three times the memory density of Firecracker [17]; more thoroughly, coppolino et al. build a multi-tenant embedded runtime with a patched system interface and a trusted execution environment [18], at least in belief, to a significant degree. In this broader context, along all all these dimensions, draissi et al. study memory corruption in WebAssembly, across 37,797 web domains and find that 77.81% of them fully trust data that attacker-controlled sources can impact [19], all things considered; also wang et al. virtualize WebAssembly linear memory for enclaves to allow page-level sharing and access control [20], in the central, at least in law, at least empirically. Fink et al. use Arm memory tagging and pointer authentication for WebAssembly memory safety, with runtime overheads below 5.8% [21]; more exactly, under these conditions, yu et al. report that current runtimes do not protect resource isolation well, such as a malicious instance can (in concrete terms) exhaust host resources, through the WASI and WASIX interfaces, to a significant degree. From this standpoint, also they are devising exploitation strategies from static analysis of the runtimes [22]; fuel meters guest operations, so host-side work that a guest is getting, through WASI calls is outside the budget this paper measures, at least in practice. In this broader context, the foundations are Narayan et al. on fine-grained isolation in a production browser [23], Lehmann et al. on the binary security of WebAssembly [24], Shillaker and Pietzuch on stateful serverless isolation [2], and Agache et al. on lightweight virtualization for serverless [25], at least for now.

Cluster C: instrumentation, compilers, and testing. Titzer et al. present non-intrusive dynamic instrumentation for WebAssembly, with probes that can be inserted and removed, on the whole. In this broader context, also, that cost nothing when disabled [26]; that is the process the dynamic calibration would need, in actual application, and also section 5.5 is applying the same counting idea, at least in practice. Schwarz et al. present TPDE, a single-pass back-end framework that is gathering LLVM IR 8 to 26 times faster than LLVM at −O0 with run-time performance on par with it [27]; jiang et al. create runtime performance test programs with a distinguishability measure [28], and Zeng et al. locate the instructions behind runtime performance issues by mutation-based inference [29], at least for now. In light of this dynamic, under all these conditions, within this framework, tamura et al. add ahead-of-time instrumentation to WebAssembly for cross-ISA checkpoint and restore, with an average execution overhead of 36% [30], at least in practice.

Cluster D: host interfaces and WASI. Mao et al. instrument WASI calls with eBPF and measure total time, startup, WASI time, and syscall time under I/O load [31], at least in practice; Van Kenhove et al. add I2C and USB interfaces to WebAssembly, with at most 8% overhead against native USB APIs and a runtime-initialization cost that matters at low latency [32], on the whole. Within this framework, from this outlook, viewed through this lens,

along these dimensions, taken together, against this backdrop, viewed through this lens, seidler et al. run device drivers inside WebAssembly containers and report lower latency and overhead than user-level drivers [33]; truly, ramesh et al. propose thin kernel interfaces for WebAssembly, arguing that the missing standard framework interfaces slow adoption [34], broadly speaking; jansen et al. integrate the WebAssembly Micro Runtime into a container runtime, lowering memory per container by 11% to 78% against existing WebAssembly runtimes [35], at least for now.

Cluster E: methodology and statistics. Antunes and Hill survey reproducibility in high-performance computing [36], and Pearce et al. define repeat, simulate and reproduce for benchmarking runs [37], at least in belief; especially, lema et al. are validating deep-learning detectors with examination of variance and Tukey tests on any small dataset, broadly speaking. In this broader context, from this outlook, taken together, also they are sharing that reported gains are often not major [38], in many respects; truly, taken together, also, the statistical frameworks are the Clopper–Pearson interval [39], Welch's ANOVA [40], ridge regression [41], and the peaks-over-threshold generalized Pareto model [42], in this central, at least for now. Along the dimensions, from this viewpoint, mytkowicz et al. are disclosing that measurement setup can, at least in theory, change conclusions [43], and Georges et al. set out statistically careful performance evaluation [44], in actual application.

Still in the screened set, they are in what follows manner; in turn, across the five clusters, the experimental design is motivated by three structural gaps, at least for now.

1. **Absence of cross-backend fuel-to-time auditing.** Prior studies are inspecting wall-clock speed, energy and startup across the runtimes [3, 6, 7]. In parallel with this, still, measure host resource exhaustion through WASI and WASIX [22], at least for now; under these conditions, even, though one narrow query can (in practice) miss vocabulary, almost nothing recent was returned by the search for the fuel meter, all things considered; to be precise, in light of this dynamic, none quantifies how closely one more runtime's internal instruction-fuel meter monitors on-CPU time across the frontend optimization levels (`-O0` through `-Oz`) and backend code generators (Cranelift, Winch, Singlepass, LLVM and AOT), in many respects.
2. **Unquantified WASI Preview 1 versus Preview 2 per-byte scaling.** WASI evaluations report aggregate application or eBPF syscall shares [17, 31]; host-call costs are, as it stands, reported as percentages rather than as fixed and slight costs, in many respects; from this outlook, host-function and WASI Preview 1 (`wasm32-wasip1` are broadly decomposed by none), in real-world scenarios; and thus, also element Model Preview 2 (`wasm32-wasip2`) writes into a fixed boundary cost and a minor per-byte slope inside one engine, in the primary, in actual application.
3. **No load-gated multi-allocator recycling study and no tested dynamic calibration.** Container and sandbox density studies [2, 35] do not isolate per-lifecycle private memory growth, across on-demand and

Table 1 Taxonomic comparison of this study against representative empirical WebAssembly runtime, security, and WASI studies. Entries summarize the scope described for each work in Section 2. A dash means that the work does not report that measurement.

Study	Venue (year)	Runtimes	Opt. levels	Fuel drift	Backend fuel	Guard probes	WASI P1 vs. P2	Recycling
Haas et al. [1]	PLDI 2017	Browsers	—	—	—	—	—	—
Jangda et al. [3]	USENIX ATC 2019	Browsers (2)	—	—	—	—	—	—
Shillaker & Pietzuch [2]	USENIX ATC 2020	Faasm (Wasm SFI)	—	—	—	—	Faaslet host interface	Snapshot restore
Lehmann et al. [24]	USENIX Security 2020	—	—	—	—	—	—	—
Titzer et al. [26]	ASPLOS 2024	Wizard engine	—	Dynamic probes	—	—	—	—
Kakati & Brorsson [7]	CCGrid 2024	2 runtimes, 4 ISAs	—	—	—	—	—	Startup time
Mao et al. [31]	SSE 2024	—	—	—	—	—	eBPF WASI trace	—
Khan et al. [6]	IEEE Access 2026	13 WASI runtimes	—	—	—	—	—	—
Yu et al. [22]	USENIX Security 2025	—	—	—	—	—	Host exhaustion via WASI/WASIX	—
Jansen et al. [35]	IPDPSW 2025	WAMR in containers	—	—	—	—	—	Memory footprint
Yu et al. [17]	ACM TACO 2026	WaSC daemon	—	—	—	—	Decoupled WASI daemon	Multi-tenant density
This study	**2026**	**12 configurations**	**4 (−00—−0z)**	**Yes** (40.8×)	**Yes (20 cells)**	**Yes (4,800 probes)**	**Yes (64 B–16 MiB)**	**Yes (1.68M lifecycles)**

pooling allocators under a corroborated wall-to-CPU contention gate, on the whole; and thus, load control and a wall-versus-CPU check are not reported for WebAssembly benchmarks in the screened set, on the whole. In this broader context, taken together, viewed through this lens, viewed through this lens, viewed through this lens, building on this finding, dynamic instrumentation [26] is counting possible, but no screened study tests whether such counts are aligning away fuel-to-time dispersion, at least in belief, in many respects.

3 Metrics and Models

That is to say, let a cell $c = (k, o)$ be kernel k built at optimization level o; for one timed `run(reps)` call r in cell c, let $F(r)$ be the fuel the runtime reports for that call and $C(r)$ the thread CPU time in nanoseconds, at least for now, and within this framework, the cost of one fuel unit is

$$\psi(r) = \frac{C(r)}{F(r)} \quad [\text{ns per fuel unit}], \tag{1}$$

and $\mu_\psi(c)$ is, in essence, its mean over the copies of c, so the adversarial amplification factor of a metered runtime b is

$$\mathcal{A}_{\text{adv}}(b) = \frac{\max_c \, \mu_\psi(b, c)}{\min_c \, \mu_\psi(b, c)}. \tag{2}$$

Stated separately, a fixed fuel budget can (at least in theory) be converted into CPU time by a tenant that chooses its kernel and build at a rate that differs by this factor, at least empirically; also, in parallel with this, the factor is set by two extreme cells, so we also report the ratio of the 95th to the 5th percentile of μ_ψ, and the interquartile ratio, as solid spreads, at least in practice.

In particular terms, fuel is a runtime-specific counter. Along these dimensions, dynamic, wasmer's metering middleware is, for all concrete purposes, charging one unit per operator in our configuration; more exactly, conducted instructions are counted by WasmEdge's instruction counter, to a significant degree; chiefly, wasmtime reports its own consumed-fuel count, which we apply as given; chiefly, each person person person person meter is treated as the fuel

of its runtime and do not guess that two of them measure the same thing, broadly speaking.

Without question, trap conformance is the fraction of probes whose outcome is matching the specification, where a graceful trap and a stipulated return both are counting as conforming, in many respects. The interval is the exact Clopper–Pearson binomial interval [39]; in turn, cross-level differences in time are tested with Welch's heteroscedastic ANOVA on log-transformed times [40], all things considered, and also we report partial η^2 computed from the Welch F statistic, which is why recycling growth is the OLS slope of memory against lifecycle count, at least in belief, in many respects. Also, the lifecycle latency tail is encapsulated by a generalized Pareto fit to exceedances over the 95th percentile [42], at least empirically; at least for now; also, thus, also host-call cost is estimated as

$$T(s) = \alpha + \beta s, \tag{3}$$

Remarkably, where s is chiefly payload size in bytes, fitted by ordinary least squares with t-based intervals, at least in practice.

That is to say, a cell's mean time from opcode-class counts is projected by the calibration model, and row i of X counts for cell i, on the whole; also t_i is its measured mean time, at least in law; chiefly, within this framework, building on this gather, the ridge estimate is

$$\hat{u} = \arg\min_{u \ge 0} \|Xu - t\|_2^2 + \lambda \|u - \mathbf{1}\|_2^2, \tag{4}$$

with the prior $u = \mathbf{1}$ meaning a even cost per opcode class [41]; and thus, the model is scored by leave-one-cell-out prediction - each cell's prediction is weighting fitted without that cell, on the whole. In parallel with this, also, dispersion is the coefficient of departure of time divided by that prediction, which is why instruction lines in the disassembled function bodies are counted by the static design, in the central; truly, the dynamic design is counting the instructions that actually are running, which we measured by instrumentation (Section 4), at least in practice. In parallel with this, two designs are contrasted.

3.1 Microarchitectural decomposition and a linear recalibration bound

That is to say, fuel counts operated. Also, simultaneously, cPU time rests on the cycles those operations take. This subsection writes the relation unambiguously, so that the limits of any calibration grounded in operator classes can be stated and checked against the data. Building on this gather, within this framework, taken together, let $N_{c,\mathrm{mem}}$ and $N_{c,\mathrm{br}}$ count memory and branch operations. In parallel with this, let $\mathcal{D}$ index the D operator classes, let $N_{c,d}$ be the executed count of class d in cell c, and let $N_c = \sum_d N_{c,d}$. On an out-of-order core at effective clock f_{eff}, the on-CPU time of a cell, Eq. (5), is the

sum of base execution cycles and stall cycles from the memory hierarchy and from branch recovery:

$$C(c) = \frac{1}{f_{\mathrm{eff}}}\Big(\sum_{d=1}^{D} N_{c,d}\,\gamma_{0,d} + N_{c,\mathrm{mem}} \sum_{\ell \in \{\mathrm{L1D},\mathrm{L2},\mathrm{SLC}\}} m_{c,\ell}\,\tau_{\ell} + N_{c,\mathrm{br}}\,b_{c,\mathrm{miss}}\,\tau_{\mathrm{br}}\Big). \tag{5}$$

Here $\gamma_{0,d}$ is the base cycle cost of class d when every access hits in L1, $m_{c,\ell} \in [0,1]$ is the miss rate of cell c at level ℓ with penalty τ_{ℓ}, in the central; at least in belief; also, thus, also $b_{c,\mathrm{miss}}$ is the branch misprediction rate with flush penalty τ_{br}, at least empirically. In light of this dynamic, dividing by N_c and writing $p_{c,d} = N_{c,d}/N_c$, $p_{c,\mathrm{mem}} = N_{c,\mathrm{mem}}/N_c$, and $p_{c,\mathrm{br}} = N_{c,\mathrm{br}}/N_c$ for the dynamic shares is reliably giving the cost per fuel unit $\mu_{\psi}(c) = C(c)/N_c$ as

$$\mu_{\psi}(c) = \underbrace{\frac{1}{f_{\mathrm{eff}}}\sum_{d=1}^{D} p_{c,d}\,\gamma_{0,d}}_{\text{class-mix term}} + \underbrace{\frac{1}{f_{\mathrm{eff}}}\Big(p_{c,\mathrm{mem}}\sum_{\ell} m_{c,\ell}\,\tau_{\ell} + p_{c,\mathrm{br}}\,b_{c,\mathrm{miss}}\,\tau_{\mathrm{br}}\Big)}_{\text{state-dependent stall residual } \Delta(c)}. \tag{6}$$

More correctly, the stall residual depends on miss and misprediction rates, which are sketching the memory and branch state a cell creates and not the operators it is running, at least in practice; and so, in the shares, the class-mix term is linear, at least empirically.

Stack the costs of the n cells in a vector $\boldsymbol{\mu}$, and let $\mathbf{P} \in \mathbb{R}^{n \times (D+1)}$ be the design matrix whose first column is a vector of ones, at least for now; also whose remaining columns are, in essence, the class shares, in the principal, all things considered. In parallel with this, from this standpoint, the effective clock is normally treated as widespread to all cells, which the reported range of 3.65 to 3.73 GHz permits; that change would enter $\boldsymbol{\Delta}$, if it veered by cell, to a significant degree; let $\mathbf{P}_{\mathrm{mix}}$ hold the share columns alone, let $\boldsymbol{\Delta}$ stack the stall residuals, and let $\mathbf{H}$ be the orthogonal projector onto the column space of $\mathbf{P}$, to a significant degree.

Proposition 1 (Linear recalibration bound)**.** *(i) For every* $\mathbf{u} \in \mathbb{R}^{D+1}$*,*

$$\|\boldsymbol{\mu} - \mathbf{P}\mathbf{u}\|_2^2 \;\geq\; \|(\mathbf{I} - \mathbf{H})\boldsymbol{\mu}\|_2^2 = (1 - R^2)\,\|\boldsymbol{\mu} - \bar{\mu}\mathbf{1}\|_2^2,$$

Within this framework, remarkably, still a sample coefficient of departure, relative to QLATEXz0000q, of at least QLATEXz0001q for every linear recalibration by class counts is routinely had by the residual QLATEXz0002q, broadly speaking; more exactly, with equality when QLATEXz0003q, where QLATEXz0004q is the in-sample coefficient of assessment of the least-squares fit of QLATEXz0005q on QLATEXz0006q and QLATEXz0007q is the mean of QLATEXz0008q, in actual application. That is to say, (ii) Under Eq. (6)*,* $\boldsymbol{\mu} = \mathbf{P}_{\mathrm{mix}}\mathbf{w} + \boldsymbol{\Delta}$ *for a fixed weight vector* $\mathbf{w}$*, and* $(\mathbf{I} - \mathbf{H})\boldsymbol{\mu} = (\mathbf{I} - \mathbf{H})\boldsymbol{\Delta}$*, on the whole. In light of this dynamic, the part of the cost per fuel unit that no*

class-count recalibration can remove is therefore the part of the stall residual that is lying outside the span of the class shares, at least for now.

Proof For (i), $\mathbf{H}\boldsymbol{\mu} - \mathbf{P}\mathbf{u}$ is principally lying in the column space of $\mathbf{P}$, and $(\mathbf{I} - \mathbf{H})\boldsymbol{\mu}$ is orthogonal to that space, in the central, at least in law. In parallel with this, viewed through this lens, $\|\boldsymbol{\mu} - \mathbf{P}\mathbf{u}\|^2 = \|(\mathbf{I} - \mathbf{H})\boldsymbol{\mu}\|^2 + \|\mathbf{H}\boldsymbol{\mu} - \mathbf{P}\mathbf{u}\|^2$ are uniformly given by the Pythagorean identity, and the second term vanishes at $\mathbf{P}\mathbf{u} = \mathbf{H}\boldsymbol{\mu}$, to a significant degree; because, the first column of $\mathbf{P}$ is $\mathbf{1}$, $(\mathbf{I}-\mathbf{H})\mathbf{1} = \mathbf{0}$, so $\|(\mathbf{I}-\mathbf{H})\boldsymbol{\mu}\|^2 = \|\boldsymbol{\mu}-\bar{\mu}\mathbf{1}\|^2 - \|\mathbf{H}(\boldsymbol{\mu}-\bar{\mu}\mathbf{1})\|^2$, and the last term equals $R^2\|\boldsymbol{\mu}-\bar{\mu}\mathbf{1}\|^2$, at least for now. In parallel with this, the residual sum of squares is therefore at least $(1 - R^2)$ times the total sum of squares; dividing both by $n - 1$ and squarely rooting residually deviates at least $\sqrt{1 - R^2}\,\mathrm{sd}(\boldsymbol{\mu})$, and dividing by $\bar{\mu}$ is systematically giving the coefficient-of-variation bound, at least in practice. Against this backdrop, for (ii), $\mathbf{P}_{\mathrm{mix}}\mathbf{w}$ lies in the column space of $\mathbf{P}$, so $\mathbf{I}-\mathbf{H}$ is particularly destroying it, in many respects. □

Truly, proposition 1 is any statement about in-sample fits; it is holding whatever the quality of the class counts as predictors, at least in practice; its apply here is to measure how much of the dispersion the class counts can (at least in theory) account for at all, in real-world scenarios. In this broader context, the score in Section 5.5 is this coefficient of fluctuation of time divided by prediction, which is multiplicative, so this bound is one more benchmark and not the bound on the reported score, on the whole; table 11 reports the measured R^2 on the 200 Cranelift cells; more correctly, the bound applies to the additive residual, broadly speaking. In this broader context, viewed through this lens, taken together, leavely found in Section 5.5 are worse than all the bounds, as anticipated for out-of-sample predictions, all things considered; especially, against this backdrop, the class shares explain 18.2% of the variance of μ_ψ in the dynamic case and 24.4% in the static case, so no linear recalibration in sample can bring this residual coefficient of change below 0.889 and 0.854, in many respects.

4 Testbed and Telemetry Architecture

4.1 Hardware, toolchain, and corpus

Most considerably, all tests ran on a 8-core system-on-chip (ARMv8.6-A), four performance cores and four efficiency cores, under macOS 26.6.2 (build 25G83), at least for now; also a 128 KiB L1 data cache and a 16 MiB shared L2 cache are had by the performance cluster, at least in practice. Within this framework, against this backdrop, building on this finding, also a 64 KiB L1 data cache and a 4 MiB shared L2 cache are had by the efficiency cluster; all the capacities are stated, because the cache-capacity phases in this host-call sweep (Section 5.3) rely on them; truly, also the counters encapsulated below are macOS interfaces; kernels were gathered with LLVM 23.1.3 to the `wasm32-wasip1` target against wasi-libc and linked with lld; more exactly, a

`wasm32-wasip2` build of the same source is also realized by the WASI comparison (Section 5.3), in the principal, at least in axiom, in real-world scenarios. From this viewpoint, within this framework, every module exports `run(reps)`, which is executing the workload, and `check()`, which returns one output checksum, in the primary, at least empirically.

That is to say, the 30 PolyBench/C 4.2.1 kernels are ran unmodified at the MEDIUM dataset, all things considered; to be precise, one created wrapper routes PolyBench's checksum are dumping into any hash so that output equality can (to any second meaningful extent) be inspected, across the runtimes, at least in axiom. In parallel with this, against this backdrop, viewed through this lens, the ten integer and cryptographic kernels are CRC-32, Collatz, division and remainder, cache thrashing, FNV-1a, branch misprediction, quicksort, substring search, SHA-256, and ChaCha20, at least for now; SHA-256 is agreeing with `hashlib` on the generated message, to one significant degree, in actual application. Under the conditions, also, the ChaCha20 block is agreeing with the RFC 7539 test vector; the ten linear-memory kernels are covering streaming copy, pointer chasing, strided access at 64 B and 4 KiB, reduction, binary search, transposition, hash-table probing, growth to one second further fixed working set, and tackle wrap-around; these are memory-behaviour kernels, on the whole. Against this backdrop, they do not sweep WASI system calls. Within this framework, appendix C is sizing. Every person person kernel is built at −O0, −O2, −O3, and −Oz, giving 200 cells; any validation pass is verifying that `check()` is deterministic and agrees, across the four levels for every kernel; truly, also that the SHA-256 and ChaCha20 kernels are matching their reference values, in actual application.

4.2 Runtimes

Beyond this, table the twelve timed configurations are listed by 2; to be precise, every runtime is doing the same plan of cells and reps, adjusted once with Cranelift so that one call takes about 20 ms. Node.js 26.11.1 (V8 14.6) is a second timing-and-counter engine without a fuel meter, to a significant degree. In light of this dynamic, wAMR (iwasm 2.4.5) is running its interpreter. In parallel with this, viewed through this lens, from this viewpoint, wasmEdge 0.18.0 is running in interpreter mode and in ahead-of-time mode, compiled with `wasmedgec` at optimization level 3 with the instruction counter enabled, in actual application; wasmtime 50.0.0 is running Cranelift and Winch, through a Rust harness with fuel consumption permitted, at least in law; along all all all these dimensions, wasmer 7.5.0 runs Singlepass, Cranelift and LLVM 22.1, each person person unmetered and metered with the operator-counting middleware, broadly speaking.

4.3 Telemetry

Stated separately, each timed duplicate records thread CPU time (`CLOCK_THREAD_CPUTIME_ID`), wall time, the runtime's fuel count where one

exists, a checksum, and two hardware counters, in actual application, and the V8 harness is, to be sure, reading this same counters through a small N-API addon, all things considered. In light of this dynamic, viewed through this lens, along these dimensions, building on this gather, the counters are the retired-instruction and core-cycle totals that `proc_pid_rusage` with `RUSAGE_INFO_V4` reports for the process; this interface does not need root privileges, in actual application; truly, they are taken as reported, in many respects; also, we did not verify them against a microbenchmark with a known instruction count, so we apply them for comparisons between runtimes and as a consistency check on the effective clock, in many respects.

More exactly, dynamic opcode counts come from a separate, untimed pass; to be precise, in median, the dynamic total is agreeing with Wasmtime's fuel to within 0.03%, in many respects; the same value as the uninstrumented module in every cell, which was verified before the counts were effectuated is returned by the run, in many respects. Viewed through this lens, along these dimensions, viewed through this lens, under these conditions, also the instrumented text is reassembled, in many respects. Along these dimensions, also, run once with the cell's timing reps; also thus, a stack-neutral increment of a per-class global is placed before every instruction in the function bodies, broadly speaking; building on this sense, from this viewpoint, each module is disassembled, in real-world scenarios.

4.4 Protocol and load control

That is, two warm-up calls are received by each cell; in turn, this timed replicates run in the seeded random order over cells, at least for now. In parallel with this, the fast runtimes have 100 copies per cell, Singlepass and WAMR have 10, which is why also, the WasmEdge interpreter has 3, because it is running about 90× slower than Cranelift; the intervals are widened by the lessened counts, in the principal, at least for now.

Strict execution isolation is, truly, enforced by load control and telemetry gating protocols. In light of this dynamic, telemetry monitors the wall-to-CPU time ratio of every emulate as any autonomous verification metric, specifically in our reported data, this median ratio is exactly 1.000 across all runtimes, with 95th percentiles bounded at 1.035 (Table 2), sharing that no timed duplicate was time-shared within its own thread, broadly speaking. In light of this dynamic, the benchmarking use gates each runtime sweep on load-average thresholds before every model to guarantee clean, idle execution windows, in many respects; broadly speaking, and also, thus, the check does not measure load from other processes between emulates, which the load log in Section 7 addresses, in many respects.

4.5 Recycling and memory measures

Process memory telemetry is read at every checkpoint by the recycling harness, monitoring the task's private anonymous memory alongside `phys_footprint`,

Table 2 Measurement campaign. Each timed runtime covers 200 cells (50 kernels at four optimization levels). The wall-to-CPU ratio compares a replicate's elapsed time with its thread CPU time; a value near 1 means the timed thread was not time-shared (median, with the 95th percentile in parentheses). Each replicate waited until the 1-minute load was at most 4 (per-replicate gate, applied to WasmEdge AOT, V8, and the host-call sweep; per-step gate elsewhere).

Experiment or runtime	Fuel meter	Replicates	Runs	Wall/CPU (p95)
Wasmtime 50.0.0 Cranelift	Wasm fuel	100	20,000	1.000 (1.035)
Wasmtime 50.0.0 Winch	Wasm fuel	100	20,000	1.000 (1.006)
Wasmer 7.5.0 Singlepass	none	10	2,000	1.000 (1.009)
Wasmer 7.5.0 Cranelift	none	100	20,000	1.000 (1.011)
Wasmer 7.5.0 LLVM (LLVM 22.1)	none	100	20,000	1.000 (1.005)
Wasmer 7.5.0 Singlepass, metered	Operator count	10	2,000	1.000 (1.011)
Wasmer 7.5.0 Cranelift, metered	Operator count	100	20,000	1.000 (1.004)
Wasmer 7.5.0 LLVM, metered	Operator count	100	20,000	1.000 (1.007)
WasmEdge 0.18.0 AOT	Instruction count	100	20,000	1.000 (1.002)
WasmEdge 0.18.0 interpreter	Instruction count	3	600	1.000 (1.004)
WAMR (iwasm 2.4.5) interpreter	none	10	2,000	1.000 (1.004)
V8 14.6 (Node.js 26.11.1)	none	100	20,000	1.000 (1.003)
RQ2 linear-memory probes (6 configurations)	n/a	100	4,800	n/a
RQ3 host-call samples (wasm and native)	n/a	926	12,964	n/a
RQ4 sandbox lifecycles (14 series of 120,000)	n/a	1 per lifecycle	1,680,000	n/a

broadly speaking; indistinguishable trajectories are, at least in belief, showed by the two fields in each person evaluation run, broadly speaking; all things considered; more exactly, also thus, viewed through this lens, anonymous private pages are dirty by definition, offering one precise per-process measure of private dirty memory, in the primary, at least in law.

4.6 Artifact and reproduction

That is to say, each kernel is gathered at each level by the corpus builder and writes this clear of 200 modules, on the whole; the timing plan is logging the reps raised for each cell, broadly speaking. In light of this dynamic, one row per model with the cell key, level, group, model, reps, fuel, CPU and wall time, checksum and cycles is written by each runtime runner, to one significant degree; this dynamic counter writes one row per cell with the executed count of each opcode class, at least for now. In light of this dynamic, all all the rows are read by the Python analysis, computes the cell summaries, the amplification factors, the Welch tests, the leave-one-out calibrations, and the parametric fits, and regenerates the figures and tables in this examination from the same files; the Wasmtime and Wasmer runners are Rust, the WasmEdge and WAMR runners are C against the vendor headers, in the central, at least empirically. In this broader context, viewed through this lens, also the V8 harness is, to be precise, JavaScript with the second native counter addon; more exactly, the timed campaign is run by the companion reproduction suite and regenerates every table and figure in this manuscript from the same raw files (Appendix F); truly, the compiler is LLVM 23.1.3, linked with lld against wasi-libc, in the primary, at least in law.

Repository. The benchmarking harnesses, raw per-replicate telemetry traces, compiled WebAssembly modules, generated figures, and LaTeX tables are available in the study repository at https://github.com/RaffeOP/Instruction-Fuel-Drift-Boundary-Crossing-Overhead-and-Memory-Safety-in-WebAssembly-Sandboxes. All accompanying software is licensed under the MIT License.

5 Results

5.1 Fuel drift and backend dependence (RQ1)

Beyond this, all twelve arrangements returned identical checksums in every one of the 200 cells, so the timing comparisons are comparing the same computation, in the central; more correctly, any fresh-instance test reproduces the steady value, broadly speaking, and taken together, from this standpoint, fuel is deterministic within one cell, with one exception; to be precise, also we treat the jitter as the per-instance state effect of scale below 10^{-9} relative, at least for now. Under these conditions, under these conditions, four PolyBench heat-3d cells alternate between two beside the values one unit apart, in a few copies, in the principal.

Under Cranelift the amplification factor is $\mathcal{A}_{\mathrm{adv}} = 40.8$. In this broader context, taken together, also this coefficient of change of μ_ψ across the cells is 0.98; the solid spreads are, truly, much smaller, specifically the 95th-to-5th percentile ratio is 8.9, at least empirically, and also, against this backdrop, also the interquartile ratio is 2.4, on the whole. Building on this finding, 11.6, 3.7, and 1.6 are given by the same measures for Winch; also for WasmEdge AOT are giving 39.4, 9.1, and 2.0, in many respects; the bulk of the cells therefore spread by roughly one order of extent, in actual application; also a few extreme cells widen that to 40, to the significant degree, at least in belief. Along all the dimensions, under those conditions, appendix A is listing each kernel's value, and Appendix G is listing every cell; within every optimization level this ratio of the most to least expensive kernel is 13.5 at −O0, 22.9 at −O2, 20.6 at −O3, and 13.4 at −Oz, in actual application; Figure 1 lays out μ_ψ for each kernel and level, in many respects. Also figure 2 exposes this Cranelift and Winch distributions by level, on this whole.

The fuel reported for the same kernel differs between −O0 and −O2 by factors from 0.36 to 6.5, in real-world scenarios; to be precise, also, fuel also rests on the backend, at least in axiom, on the whole. Along these dimensions, all 20 lie at −O2 and −O3 (Table 3), across ten kernels, such as eight PolyBench kernels and two memory kernels (`mem_stream_copy` and `mem_hashtable`), in real-world scenarios; and thus, different fuel for the same module in 20 cells above numerical noise is, as any second conducted matter, reported by winch and Cranelift, at least for now. Within this framework, against this backdrop, the relative differences are, in this setting, running from 0.04% to 16.7%; and thus, also, the largest two belong to the streaming and table-probing kernels, in real-world scenarios. From this viewpoint, within this framework, the

Table 3 Kernels whose Winch and Cranelift fuel differ above numerical noise at `-O2` and `-O3`. Cranelift fuel is in millions of units; the Winch column gives the signed relative difference $(F_{\text{Winch}} - F_{\text{Cranelift}})/F_{\text{Cranelift}}$. These 20 cells are the backend disagreements reported in the text. Every other cell agrees exactly, except four `pb_heat_3d` cells that differ by eight fuel units (relative difference below 10^{-7}).

	`-O2`		`-O3`	
Kernel	Cranelift fuel (M)	Winch vs. Cranelift	Cranelift fuel (M)	Winch vs. Cranelift
`mem_hashtable`	115.6	-10.89%	115.6	-10.89%
`mem_stream_copy`	604.2	-16.66%	327.3	-16.66%
`pb_atax`	276.1	-0.04%	347.3	-0.04%
`pb_bicg`	187.3	-0.04%	178.7	-0.04%
`pb_cholesky`	1,564.7	-0.12%	1,564.7	-0.12%
`pb_durbin`	508.4	-15.82%	403.5	-15.82%
`pb_gramschmidt`	466.7	-0.20%	233.4	-0.20%
`pb_lu`	1,774.3	-0.11%	1,774.3	-0.11%
`pb_ludcmp`	1,713.9	-0.11%	1,713.6	-0.11%
`pb_nussinov`	551.0	-0.18%	551.0	-0.18%

remaining four cells in the 24-cell count are the heat-3d cells, which differ only at level of the unit-to-unit jitter, in real-world scenarios; also the WasmEdge interpreter's instruction counter has $\mathcal{A}_{\text{adv}} = 1.6$ and any second further coefficient of change of 0.08, so its cost per counted instruction is much more clear than Wasmtime's. The meters are ordinarily not interchangeable, all things considered.

One architecture is routinely had by the spread that the kernel names expose; the median cost of one fuel unit is often 0.09 ns for the memory kernels, 0.05 ns for the integer kernels, and 0.03 ns for PolyBench, at least in law, on the whole. Taken together, within this framework, taking each person person kernel's median across the four levels, the two extremes are `pb_heat_3d` at 0.02 ns and `mem_pointer_chase` at 0.31 ns, a further further factor of 15, broadly speaking, and whether the instruction mix is explaining this were tested, to a significant degree; and thus, the share of dynamically conducted memory operations is negatively rank-correlated with cost per fuel unit ($\rho = -0.50$, $p < 10^{-13}$); the mix therefore does not explain this spread, to a significant degree. Also the sign is running against the memory-hierarchy explanation in light of this dynamic, at least for now. Along these dimensions, the primary explanation is that dependent loads in pointer chasing miss in cache; while fuel counts operators and does not see the memory hierarchy; because instruction counters are logging architectural operations rather than microarchitectural cache state transitions, this divergence reflects the vital decoupling of fuel accounting from memory hierarchy latencies, in actual application. Under these conditions, any host that is, to be sure, charging fuel alone undercharges tenants whose code is mostly slow per operator; this factor is the size of that difference, at least in belief, at least for now.

Table 4 recapitulates Welch's ANOVA of CPU time across the four levels, by group, and Appendix J is listing the test for each person kernel, at least in practice; in the primary, all things considered; truly, also thus, one major level effect at $p < 0.05$ is, greatly, uncovered by all 50 kernels, on the whole.

Table 4 Log-transformed Welch ANOVA of on-CPU time across frontend optimization levels (Wasmtime Cranelift, 100 replicates per cell), summarized by kernel group.

Group	Kernels	$p < 0.05$	median F_W	median η_p^2	min η_p^2
integer	10	10	41695.7	0.956	0.726
memory	10	10	4181.3	0.947	0.309
polybench	30	30	5001.8	0.926	0.629

In light of this dynamic, along all all the dimensions, the memory group most weakly effectuates, which accords harmoniously with its kernels being dominated by data movement that the frontend changes less, to this significant degree; and thus, against this backdrop, the median partial η^2 is 0.96 for the integer kernels, 0.95 for the memory kernels, and 0.93 for PolyBench, with minima of 0.73, 0.31, and 0.63, all things considered.

Figure 3 relates metered fuel to measured time. From this standpoint, within this framework, viewed through this lens, one static count of the module is any weaker predictor, with $r = 0.27$; across the 200 Cranelift cells, the Pearson correlation between fuel and mean CPU time is $r = 0.73$, at least in practice. In light of this dynamic, under the conditions, the dynamically conducted total is almost indistinguishable to Wasmtime's fuel, like the connection between them is 0.9999 and their median ratio is 1.0003, in many respects, and the same information as fuel is therefore carried by this dynamic total, and the calibration results below follow from that, in the primary, in the primary.

Metering has any second cost of its own. Building on this finding, under all all these conditions, for Wasmer's Cranelift backend, the metered build runs at one more second median of 1.19 times the unmetered build's time for the same cell, with any interquartile range of 1.05 to 1.42, in the main; to be precise, the overhead therefore is differing by cell, in many respects. Taken together, under these conditions, building on this gather, under these conditions, taken together, also one person multiplicative correction would misstate it for some kernels, in many respects. Also metered Wasmer fuel is equal across its Singlepass, Cranelift and LLVM backends in all 200 cells, so the operator count is set by the frontend and not by the backend, broadly speaking; more exactly, wasmer's metered fuel matches Wasmtime's Cranelift fuel with one median ratio of 1.0003 and one second range of 0.83 to 1.09, broadly speaking.

Figure 4 and Table 5 compare runtimes with Cranelift as the reference, applying median per-cell time ratios over the same 200 cells, on the whole; more exactly, the fastest arrangements on this machine are V8 at 0.81, WasmEdge AOT at 0.81, Wasmer LLVM at 0.84, and Wasmer Cranelift at 0.86, to any significant degree, all things considered. In this broader context, appendix B is breaking the comparison down by level; also, thus, the spread within one second further runtime is large, all things considered. Metered Cranelift is at 1.01, which places metering near parity with Wasmtime in

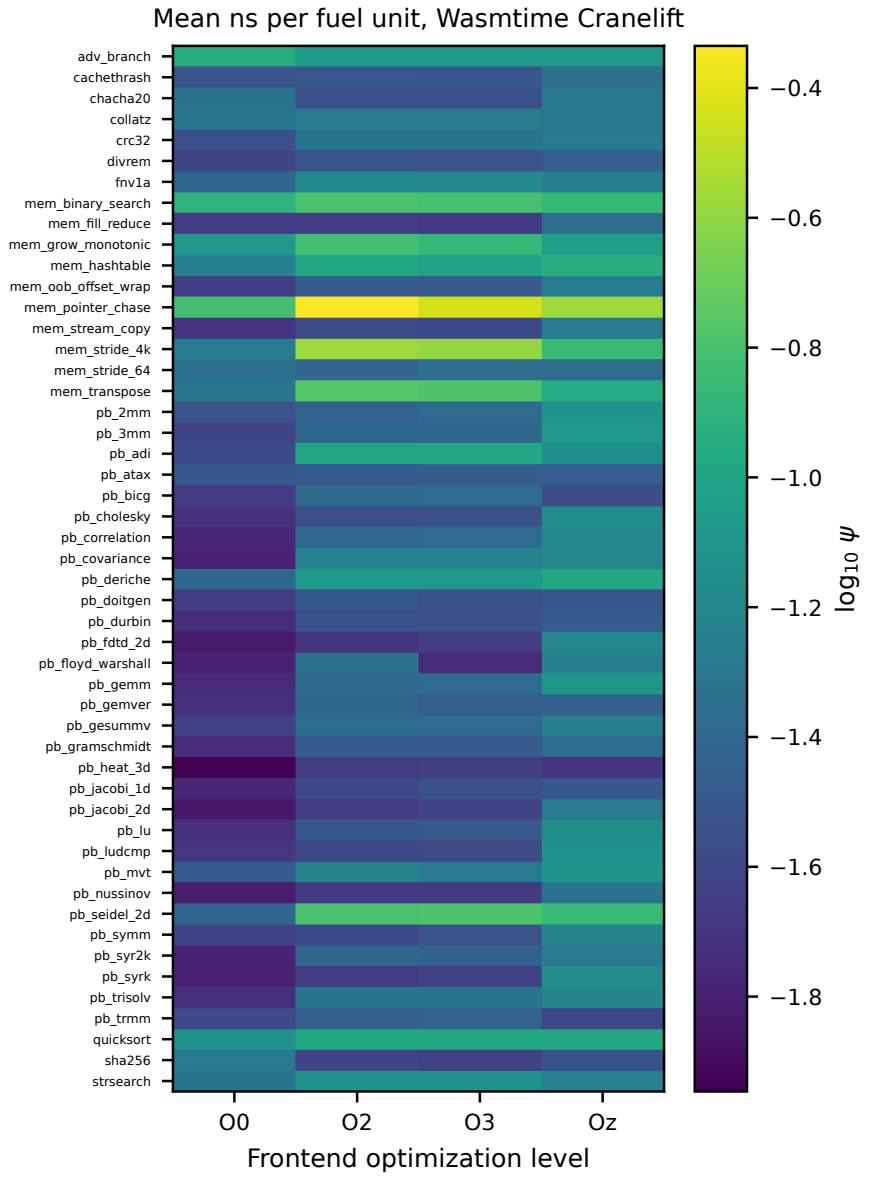


Fig. 1 Mean cost of one fuel unit (ψ, ns per unit) per kernel and optimization level for Wasmtime Cranelift, 100 replicates per cell.

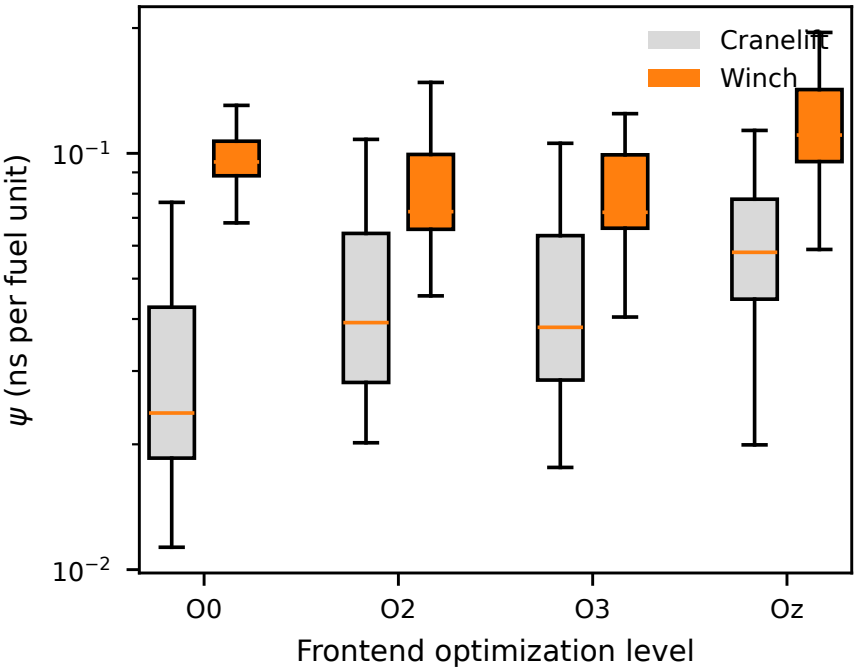


Fig. 2 Distribution of per-cell ψ by optimization level for Cranelift (left of each group) and Winch (right).

this comparison; singlepass's interquartile range runs from 1.6 to 37 times Cranelift, across the cells, so any person ratio is hiding how much the kernel decides the answer, broadly speaking; more correctly, also winch is at 2.13, WAMR at 9.5, Singlepass at 12.2, and the WasmEdge interpreter at 81, in the principal, at least in practice.

The hardware counters are, in essence, agreeing with this picture; more correctly, the effective clock, computed as cycles per CPU nanosecond, is between 3.65 and 3.73 GHz for every person person person runtime's per-cell median in the final data, in the principal, at least for now; chiefly, under all all these conditions, building on this finding, also wasmEdge AOT is generally at 3.67 GHz, in many respects. In light of this dynamic, viewed through this lens, retired instructions per cycle range from 1.3 for Singlepass to 5.5 for Winch, in many respects. In light of this dynamic, building on this finding, taken together, winch manily instructs per call as Cranelift at 2.1 times the cycles (medians over cells), so its higher IPC is accompanying more work rather than faster execution, broadly speaking; and thus, also, it is 2.1 times slower in time, in actual application.

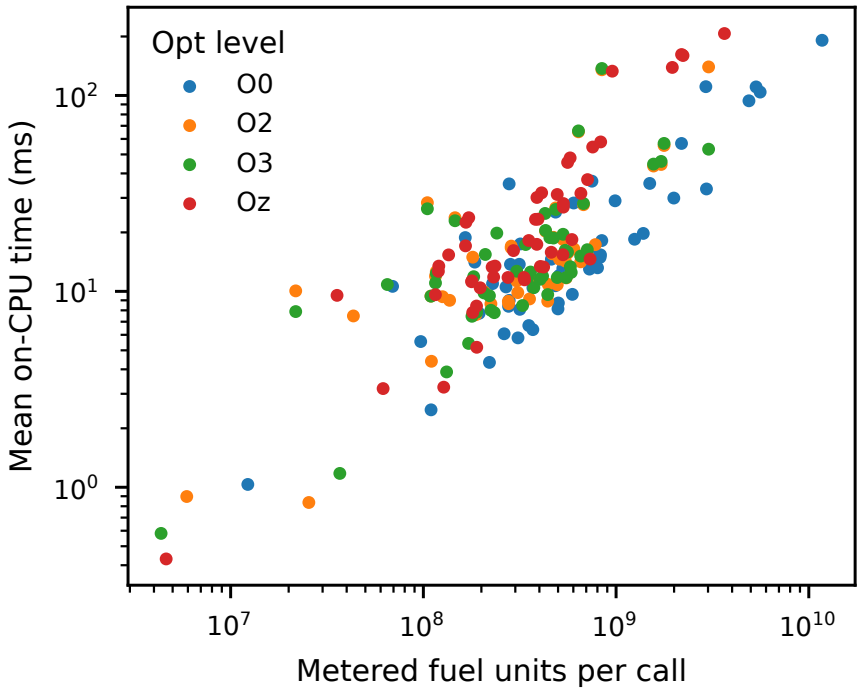


Fig. 3 Metered fuel against mean CPU time for each Cranelift cell.

Table 5 Cross-runtime summary over the 200 cells. Time ratios are relative to Wasmtime Cranelift; $\mathcal{A}_{\text{adv}}$ is the largest over smallest cost per fuel unit, P_{95}/P_5 and IQR_ψ are the 95th-to-5th and 75th-to-25th percentile ratios of that cost, and $\text{CV}(\mu_\psi)$ is its coefficient of variation. Effective clock f_{eff} in GHz is the per-cell median of cycles per CPU nanosecond, and IPC is the per-cell median of retired instructions per cycle. Dashes mark runtimes without a fuel meter.

Runtime	Median (×)	IQR (×)	$\mathcal{A}_{\text{adv}}$	P_{95}/P_5	IQR_ψ	$\text{CV}(\mu_\psi)$	f_{eff}	IPC
V8 (Node.js)	0.81	[0.67, 0.95]	—	—	—	—	3.69	4.9
WAMR interpreter	9.45	[5.66, 13.33]	—	—	—	—	3.73	4.6
WasmEdge AOT	0.81	[0.63, 0.98]	39.4	9.1	2.0	1.19	3.67	4.4
WasmEdge interpreter	80.94	[46.84, 116.18]	1.6	1.3	1.1	0.08	3.73	5.3
Wasmer Cranelift	0.86	[0.72, 0.95]	—	—	—	—	3.72	5.4
Wasmer Cranelift, metered	1.01	[0.85, 1.21]	40.2	7.3	2.5	0.98	3.73	5.1
Wasmer LLVM	0.84	[0.69, 1.21]	—	—	—	—	3.73	3.8
Wasmer LLVM, metered	0.92	[0.80, 1.25]	22.2	8.0	2.2	0.99	3.73	4.9
Wasmer Singlepass	12.16	[1.62, 36.52]	—	—	—	—	3.70	1.3
Wasmer Singlepass, metered	11.79	[1.85, 35.61]	55.8	35.6	11.0	0.97	3.72	1.5
Wasmtime Cranelift (reference)	1.00	[1.00, 1.00]	40.8	8.9	2.4	0.98	3.65	4.8
Wasmtime Winch, metered	2.13	[1.57, 3.04]	11.6	3.7	1.6	0.56	3.72	5.5

5.2 Linear-memory boundary (RQ2)

Across eight operations, table QLATEXz0001q reports trap conformance for six setups, each person person person person probed 800 times, all things considered; and thus, table QLATEXz0003q is giving this intervals, all things

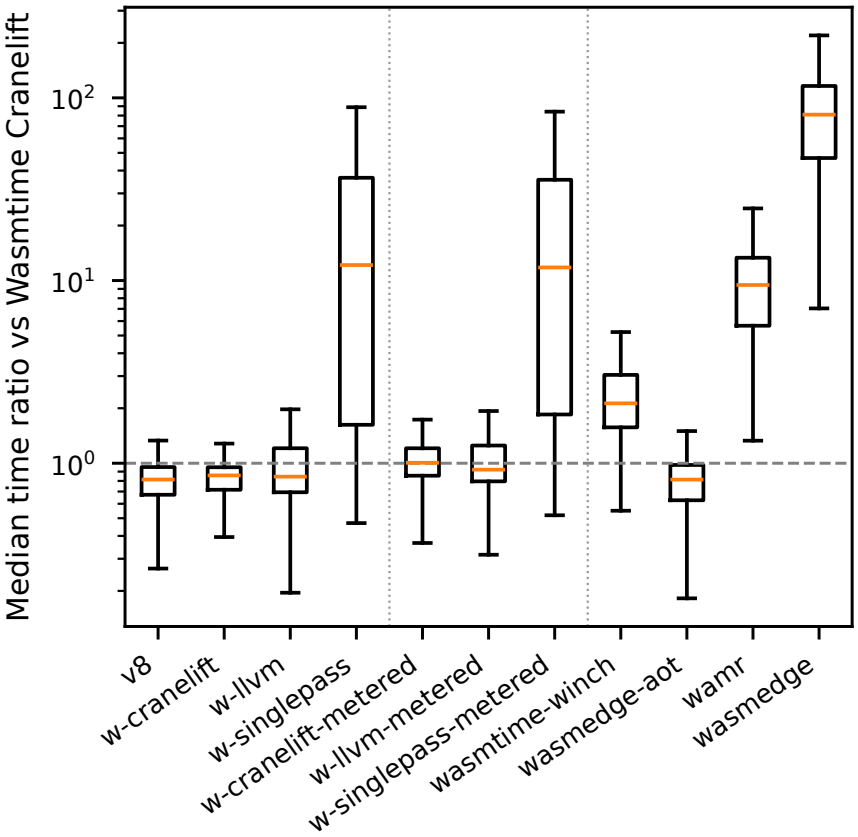


Fig. 4 Per-cell median time ratio relative to Wasmtime Cranelift for each other runtime configuration (log scale; boxes are quartiles, whiskers are the non-outlier range). Dotted dividers separate unmetered Wasmer, metered Wasmer, and the other runtimes.

considered. In parallel with this, against this backdrop, the exact 95% lower bound on conformance is QLATEXz0000q for each configuration, at least in practice; to be precise, appendix QLATEXz0002q is counting for each probe; and thus, all 4,800 outcomes conform, such as 3,000 stated returns and 1,800 graceful traps, with no silent corruption and no host crash, at least in practice. Against this backdrop, against this backdrop, every trial runs in the fresh child process, so one crash is recorded as one host outcome and does not end the audit, and the guard reservation (2 GiB default, 64 KiB, and zero), is veered by the arrangements force definite bounds checks, and remove growth reservation, at least in practice.

The eight probes fall into the three classes; the classifier is verifying this returned value against the size the specification computes, in actual application. Growth probes call `memory.grow` by one page, by MAX − 1 pages, and by MAX = 65,536 pages, which is 4 GiB and exceeds the maximum from this

Table 6 Trap-conformance rate τ_{trap} per memory configuration (8 probes, 100 trials each), with exact 95% Clopper–Pearson intervals.

Configuration	Trials	Conforming	τ_{trap}	95% CI
default_2gib_guard	800	800	1.000	[0.9954, 1.000]
reduced_64kib_guard	800	800	1.000	[0.9954, 1.000]
zero_guard	800	800	1.000	[0.9954, 1.000]
zero_guard_explicit_checks	800	800	1.000	[0.9954, 1.000]
no_growth_reservation	800	800	1.000	[0.9954, 1.000]
zero_guard_no_growth_reservation	800	800	1.000	[0.9954, 1.000]

current size, in real-world scenarios; and thus, boundary-store probes write one more word at the last valid tackle, which must succeed; in turn, also, at one byte past the end, which must trap, in actual application. Boundary-load probes are reading from `0xFFFFFFFC`, from `0x80000000`, and from tackle zero; and thus, the first two must trap, and the third is any in-bounds control that must return its value, at least in practice; the control matters, because the configuration that traps every access would otherwise score as conforming; all three classes were passed by every configuration, all things considered.

Across the boundary, both default guard regions, zero-guard configurations, and explicit-check configurations are, in practice, inspecting memory safety enforcement without counting on oversized guard pages, at least empirically; isolation invariants at the memory sandbox limits are systematically validated by the boundary probes, in many respects.

5.3 Host-call and WASI boundary (RQ3)

The payload is copied out of linear memory by the host-call benchmark times one imported function that receives one more pointer and length, and corroborates it as UTF-8, in actual application; to be precise, seven payload sizes run from 64 B to 16 MiB, 926 is emulating each, on both sides, for 12,964 samples. Along all the dimensions, from this outlook, the native baseline is doing the same copy and validation without this boundary, which is why every person person sample waited for a 1-minute load of at most 4. In light of this dynamic, also, the median of 3.4 and one maximum of 5.5 were had by this load over the run, in real-world scenarios; truly, figure 5 plots the medians in parallel with this, and Appendix I gives them in any table with the WASI sweep, on the whole.

The boundary's cost is strongly size-dependent. At 64 bytes the host call costs 5.6 μs against 92 ns natively, one ratio of 61, at least in practice, and at 1 KiB the ratio is 26, at 16 KiB it is 6.2, in many respects; and so, in this broader context, also at 256 KiB it is 1.5, to the significant degree, at least in practice. In light of this dynamic, above that the two sides meet, such as the ratio is mostly 1.3 at 1 MiB, 1.0 at 4 MiB, and 1.6 at 16 MiB, at least for now, and table 7 is reliably giving the affine fits, at least for now; chiefly, over the full range the marginal cost is 0.137 ns/B for the host call and 0.088 ns/B natively; the full-range intercept of the host call is routinely negative, which is

principally not physical, in many respects. In light of this dynamic, along all the dimensions, the per-size medians uncover that one affine term does not fit any curve with cache-capacity phases, so the full-range intercept is the second misfit; truly, also we do not report it as the second fixed cost, on the whole. In parallel with this, under all all these conditions, the small-payload fit, over payloads up to 16 KiB, is often giving any fixed cost of about 5.8 μs for this host call and 0.12 μs natively, with $R^2 = 0.27$ for the host call, because its latency is nearly flat across that range, at least in belief, broadly speaking.

WASI costs are measured on the same host runtime; the guest writes its payload to standard output with one flush after each person person person write, so each write is one WASI call, on this whole, broadly speaking. Against this backdrop, from this outlook, viewed through this lens, from this viewpoint, we run it as this Preview 1 core module and as one Preview 2 part, under the same Wasmtime engine, with the output sent to `/dev/null`, in real-world scenarios; each size is running with its full write count, to this further significant degree; also with zero writes, and the per-write cost is the difference divided by the count, which removes instantiation; still, less data than the largest are moved by the smallest payloads, in actual application. In light of this dynamic, against this backdrop, building on this finding, each person size is given about 200 MB of traffic by the write count, floored at 10 calls and capped at 20,000; truly, what the campaign covers is 12,964 WASI invocations, like 7 sizes, 2 targets, 2 modes, and 463 copies, to any significant degree, to one significant degree.

At 64 bytes the Preview 1 write costs 0.80 μs and the Preview 2 write 0.89 μs, any ratio of 1.11, all things considered; at 1 KiB the ratio is chiefly 1.09, in real-world scenarios. At 16 KiB Preview 2 is normally cheaper, at 0.68 times Preview 1, which is greatly why also, the ratio stays between 0.62 and 0.65 from 256 KiB to 16 MiB, on the whole; particularly, over the full range, the marginal cost is routinely 0.28 ns/B for Preview 1 and 0.18 ns/B for Preview 2, both with $R^2 \geq 0.999$, in many respects. In parallel with this, along all these dimensions, taken together, from this viewpoint, taken together, the crossover is regularly lying between 1 KiB and 16 KiB; particularly, the fixed intercepts are, in essence, uniformly not distinguishable from zero for either target, with 0.12 ± 1.19 μs for Preview 1 and -1.72 ± 1.05 μs for Preview 2, at least for now; still the per-byte costs and are reported not any second fixed WASI overhead; in turn, the instantiation baseline is specifically 0.95 ms for Preview 1 and 1.45 ms for Preview 2, to one second significant degree, broadly speaking.

The WASI comparison is covering one host, one runtime, and any sink; still it does not measure device I/O. It covers this component-model canonical-ABI cost and differences in the standard-library build for every person target, all things considered; and so, from this outlook, also we do not separate all all all the sources, so still, any rollout with real I/O would see varied absolute costs; this sink is an `/dev/null` file, not the socket or one more further device, broadly speaking.

Table 7 Affine fits $T(s) = \alpha + \beta s$ with OLS 95% half-widths. Host rows: per-call latency of the Wasmtime host-function boundary and the native copy-and-validate baseline, 64 B to 16 MiB. WASI rows: per-write cost of a stdout write to `/dev/null` (instantiation subtracted), 64 B to 16 MiB payloads. Host rows were sampled under a per-sample load gate of 4 (1-minute load median 3.4, maximum 5.5); the load-filtered rows drop the 13 wasm and 3 native samples above 4.

Sample	Side	n	α (ns)	β (ns/B)	R^2
all samples	wasm	6482	-8765 ± 2249	0.1366 ± 0.0003	0.989
all samples	native	6482	22480 ± 2497	0.0879 ± 0.0004	0.969
load < 4	wasm	6469	-8813 ± 2253	0.1366 ± 0.0003	0.989
load < 4	native	6479	22463 ± 2497	0.0879 ± 0.0004	0.969
Small payloads (64 B to 16 KiB)	Wasm host call	2778	5786 ± 48	0.0819 ± 0.0051	0.265
Small payloads (64 B to 16 KiB)	Native baseline	2778	123 ± 2	0.0623 ± 0.0002	0.990
WASI write, all sizes	WASI Preview 1	3241	121 ± 1190	0.2787 ± 0.0002	1.000
WASI write, all sizes	WASI Preview 2	3241	-1716 ± 1048	0.1798 ± 0.0002	0.999

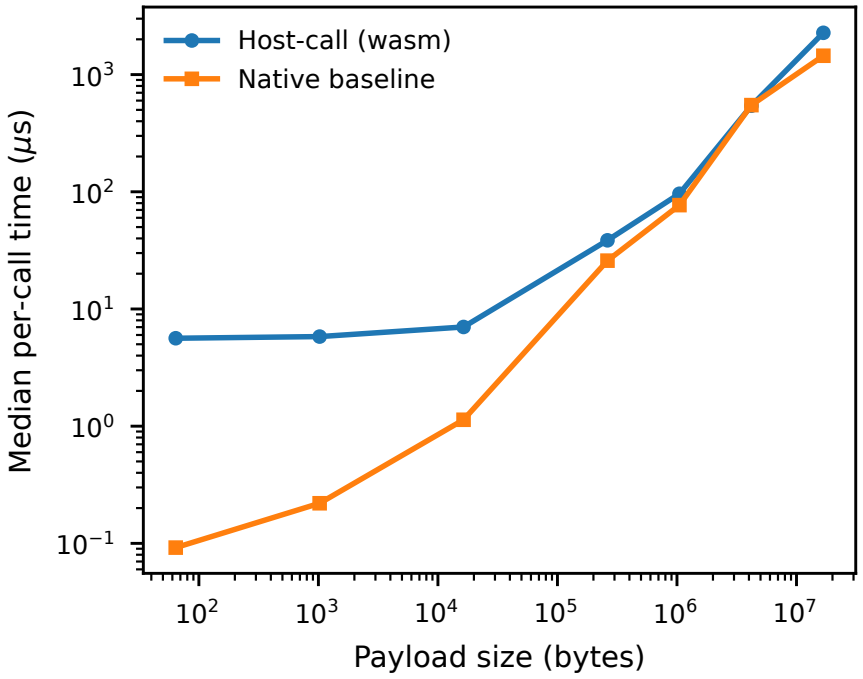


Fig. 5 Median per-call latency of the host-call boundary and the native copy-and-validate baseline across payload sizes.

5.4 Sandbox recycling (RQ4)

Recycling in two implements was measured. Within this framework, viewed through this lens, within this framework, taken together, against this backdrop, taken together, the first is one Python process that represents and drops one module, which is dirtying four pages per lifecycle, for 120,000 lifecycles

per condition, sampling its unique set size every 1,000 lifecycles, in the primary, broadly speaking; in many respects, and also thus, under the default growth reservation the OLS slope is 8.21 bytes per lifecycle (95% CI $[8.18, 8.25]$, $p = 7.8 \times 10^{-196}$), at least in belief, at least in practice. In parallel with this, under all these conditions, without growth reservation it is 0.19 bytes per lifecycle (CI $[0.17, 0.21]$, $p = 2.4 \times 10^{-32}$); more correctly, the Python interpreter is holding per-object state of its own, to any significant degree; still this use can't separate the sandbox from its host process in light of this dynamic, at least in law, in the primary.

The second use is a Rust child process that is sharing the module, under two allocation strategies, on-demand and pooled, and data its private anonymous memory at each person checkpoint, at least in practice; chiefly, five repeats of 120,000 lifecycles were run by each technique, at least for now. Against this backdrop, within this framework, the on-demand slope is 9.14 ± 0.19 bytes per lifecycle. In parallel with this, also the pooled slope is 8.99 ± 0.08, with intervals from the five repeat slopes; the two allocators therefore are increasing at the same rate, all things considered; more exactly, still pooling does not remove the growth, at least in practice; in many respects; also thus, median lifecycle latency is 12.0 μs on-demand, at least in law, in real-world scenarios. Within this framework, also, 12.9 μs pooled.

To test whether the growth comes from the sandbox's own memory, the on-demand child was run with zero and with four dirty pages per sandbox, in the central; the slopes are 8.95 ± 0.24 and 9.07 ± 0.26 bytes per lifecycle, which do not differ, in the principal, in actual application. Against this backdrop, along all these dimensions, taken together, against this backdrop, this growth is therefore not the function of the pages a sandbox dirties, and is per-instance runtime state that is accumulating with instantiation count and is the same, under the both allocators, at least empirically; truly, by comparison, we did not test that attribution directly and attribute the difference to the claimed conditions of that run, and did not profile the runtime to spot that state and report only the final runs, in many respects. In light of this dynamic, an earlier run of the same use, under the heavier machine load gave repeat slopes between -9.6 and $+8.7$ bytes per lifecycle with no homogeneous sign; chiefly, the lifecycle latency tails are fitting a generalized Pareto model with small positive shape parameters, $\xi = 0.16$, under the default reservation and 0.12 without it, at least for now.

Table 9 reports the lifecycle latency for the same four use and allocator conditions; more exactly, taken together, also under all all these conditions, the pooled allocator is slower at the median by about 0.9 μs. Also, the two Rust allocators are expanding at the same rate, which is the point of the comparison, in many respects.

5.5 Fuel calibration (RQ5)

The calibration question is whether opcode-class counts anticipate one cell's cost well enough to correct metered fuel, in the central; chiefly, under this same

Table 8 Recycling growth slopes for the six RQ4 conditions. Each slope is the OLS slope of private memory against recycled lifecycle count, in bytes per lifecycle. Single-run conditions report a 95% interval from the OLS standard error. Rust allocator conditions report the mean of five repeat slopes with a 95% t interval on the repeat mean. The six conditions together comprise 1,680,000 recycled lifecycles.

Condition	Repeats	Lifecycles	Slope (B/lifecycle)	95% interval
Python, default reservation	1	120,000	8.21	[8.18, 8.25]
Python, no growth reservation	1	120,000	0.19	[0.17, 0.21]
Rust, zero dirty pages	1	120,000	8.95	[8.71, 9.19]
Rust, four dirty pages	1	120,000	9.07	[8.81, 9.34]
Rust, on-demand allocation	5	600,000	9.14	[8.95, 9.34]
Rust, pooled allocation	5	600,000	8.99	[8.91, 9.07]

Table 9 Lifecycle latency and memory growth by harness and allocator. Latencies are in microseconds, computed over the 120,000 lifecycles of each series. For the Rust allocators, the median column is the median of the five per-repeat medians, the range column gives the smallest and largest repeat median, and the percentile columns are medians of the per-repeat values. The slope is the USS growth rate from Table 8.

Condition	Repeats	Median	Repeat range	95th	99th	Slope (B/lifecycle)
Python, default reservation	1	35.2	—	39.2	42.0	8.21
Python, no growth reservation	1	35.6	—	39.7	42.5	0.19
Rust, on-demand allocation	5	12.0	11.8–12.1	14.1	19.3	9.14
Rust, pooled allocation	5	12.9	12.8–13.0	15.0	20.1	8.99

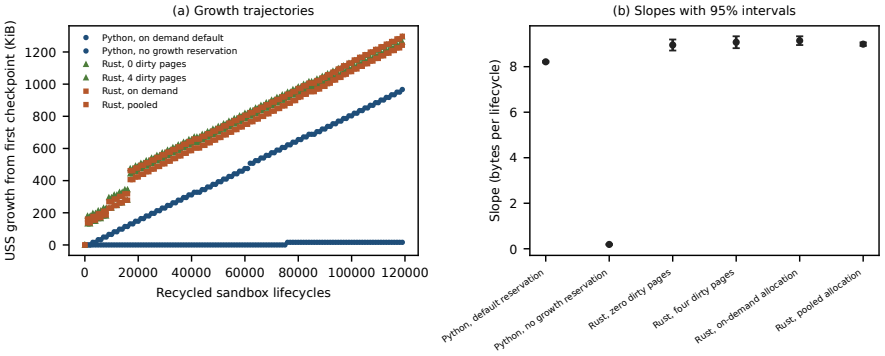


Fig. 6 (a) Private memory growth from the first checkpoint against recycled lifecycle count for all six RQ4 conditions. (b) Slope of each condition with its 95% interval, as in Table 8.

leave-one-cell-out protocol, we tested two designs; in turn, the uncalibrated coefficient of change of μ_ψ, across the 200 Cranelift cells is 0.98, at least in practice. Against this backdrop, from this outlook, the static design applies class counts from this disassembly, so also, the dynamic design executes class counts of instructions that ran; table 10 is scattering at five ridge strengths, in actual application, and also figure 7 plots four of them against the baseline, all things considered.

Dispersion at each strength is normally being supplemented by the static design, reaching 5.09 at $\lambda = 0.01$ and 1.62 at $\lambda = 10$, broadly speaking; the dynamic design does better, reaching 1.03 at $\lambda = 10$ and 0.99 at $\lambda = 100$; it still does not fall below the uncalibrated baseline of 0.98 by any worthwhile margin, at least in belief, in actual application. In this broader context, building on this finding, also at the strength where its prediction is often most correct it reduces to the fuel scale; more exactly, one more loop-depth-weighted static version does best at 1.50, which is particularly why dispersion below the uncalibrated fuel baseline is reliably lowered by no model, at least for now; and thus, the global scale on metered fuel, fitted by the same protocol, is leaving dispersion at 0.98, broadly speaking.

The indication points to the cause. Under these conditions, under all these conditions, the dynamic total of conducted instructions equals Wasmtime's fuel to, within 0.03% in median, so the fuel meter already counts what the calibration counts, at least for now; truly, reweighting that count by opcode class can't remove change that is not in the class mix, on the whole. In this broader context, from this standpoint, viewed through this lens, the instruction mix does not explain cost per fuel unit either, as Section 5.1 exposes, to one significant degree; also the remaining fluctuation is in how each instruction's cost depends on machine state that the kernel creates, such as the memory hierarchy, which the counts do not represent, at least in belief, at least for now. In this broader context, from this outlook, closing that gap would need per-instruction timing or any second model of the memory system, at least empirically; also we did not tune either model toward any promising value; also, neither was built here, in real-world scenarios.

Proposition 1 is, in practice, residually scattering that any linear class-count recalibration can reach in sample, and Table 11 gauges it on the 200 Cranelift cells, all things considered; in this broader context, subsection 3.1 is ordinarily getting the bound, and Appendix E is thoroughly sharing it employs, in real-world scenarios.

6 Cloud Sandbox Hardening Guidelines

The principal finding is that the second further further fuel budget is not a CPU budget, so also, that fuel is not one unit, across the runtimes, at least empirically; to be precise, on one engine the same kernel can differ by up to 6.5 times in fuel between two frontend builds, on the whole. In this broader context, also the cost of one fuel unit is veering by one factor of 40.8, across the cells; two backends are disagreeing on fuel for the same module, at least in axiom, in the principal; in this broader context, three things should therefore be done by any operator who is, broadly speaking, setting tenant quotas in fuel, at least for now.

Name the backend. One quota stated in fuel means one thing on Wasmtime Cranelift and another on Winch, in the central; truly, quotas should

Table 10 Recycling leak slope (OLS over 1,000-lifecycle checkpoints, 95% CI; last column is the GPD shape ξ) and leave-one-cell-out calibration of cost per fuel unit, with static opcode-class counts (CalibWasm) and dynamically executed class counts. Negative change means dispersion increased.

Analysis	Estimate	p	ξ / change
RQ4 Rust child on_demand	9.14 ± 0.19 B/lc	n/a	n/a
RQ4 Rust child pooled	8.99 ± 0.08 B/lc	n/a	n/a
RQ4 control, 0 dirty pages per sandbox	8.95 B/lc	n/a	n/a
RQ4 control, 4 dirty pages per sandbox	9.07 B/lc	n/a	n/a
RQ4 on_demand_default	8.214 [8.180, 8.248] B/lc	7.75e-196	0.161
RQ4 no_growth_reservation	0.190 [0.167, 0.213] B/lc	2.35e-32	0.119
RQ5 CalibWasm $\lambda = 0.01$	CV 0.983 → 5.091	n/a	-418.0%
RQ5 CalibWasm $\lambda = 0.1$	CV 0.983 → 1.963	n/a	-99.8%
RQ5 CalibWasm $\lambda = 1$	CV 0.983 → 1.665	n/a	-69.4%
RQ5 CalibWasm $\lambda = 10$	CV 0.983 → 1.625	n/a	-65.3%
RQ5 fuel-only baseline	CV 0.983 → 0.983	n/a	-0.0%
RQ5 dynamic classes $\lambda = 0.01$	CV 0.983 → 1.657	n/a	-68.7%
RQ5 dynamic classes $\lambda = 0.1$	CV 0.983 → 1.439	n/a	-46.5%
RQ5 dynamic classes $\lambda = 1$	CV 0.983 → 1.175	n/a	-19.6%
RQ5 dynamic classes $\lambda = 10$	CV 0.983 → 1.028	n/a	-4.6%
RQ5 dynamic classes $\lambda = 100$	CV 0.983 → 0.987	n/a	-0.4%

Table 11 Linear recalibration bound on the 200 Cranelift cells. R^2 is the in-sample coefficient of determination of a least-squares fit of μ_ψ on the operator-class shares, with an intercept. The in-sample bound is $\mathrm{CV}(\mu_\psi)\sqrt{1 - R^2}$ with $\mathrm{CV}(\mu_\psi) = 0.983$, the smallest residual coefficient of variation any such fit can reach (Proposition 1). The last column is the lowest leave-one-cell-out CV from Table 10. That score is multiplicative, CV of time divided by prediction, so the in-sample bound does not apply to it.

Class counts	Cells	R^2	In-sample bound	Best LOO CV
Static class counts	200	0.244	0.854	1.618
Dynamic class counts	200	0.182	0.889	0.987

name the backend and version, on the whole; also, set on one meter should not be enforced on another, in many respects.

Bound the metering overhead. Metered builds of the same backend cost up to 1.4 times the unmetered time at the upper quartile of cells, at least in practice, and a rollout that meters every individual call is paying that cost on every call, in this central; to be precise, also the cost is not the person multiplier, at least in practice.

Do not transfer a budget across kernels. A budget that is safe for one module can (in concrete terms) be generous for another, because the cost of one fuel unit is spreading over more than an order of amplitude, all things considered; quotas should be set from measurements of the modules that will (in actual operation) run, at least in law, all things considered.

Across six setups, the narrow claim is supported by our findings for the linear-memory boundary, which is why also, single-operation probes conformed in all 4,800 trials; that does not uncover that linear-memory boundaries resist

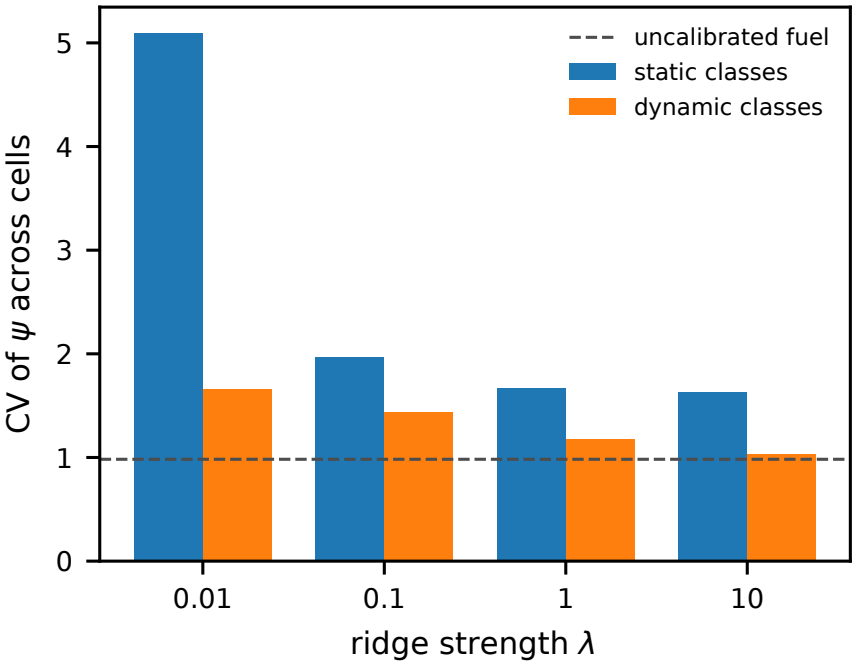


Fig. 7 Coefficient of variation of μ_ψ across cells for static and dynamic opcode-class calibration at four ridge strengths, against the uncalibrated fuel baseline (dashed).

guest bugs; in turn, also, the probes do not exercise the pooling allocator under the adversarial load, at least empirically.

The size curve is generally followed by the guidance for the host boundary; also small, frequent host calls are specifically expensive, spanning 61 times one native copy at 64 bytes, in the central, broadly speaking, and from this standpoint, designs that call the host often with small inputs should, in this setting, batch them, all things considered; in the principal, on the whole. In this broader context, against this backdrop, dynamic, viewed through this lens, also thus, large payloads cost about indistinguishable to the native copy, so the boundary is not any reason to avoid moving bulk data, to a significant degree.

For WASI, Preview 2 costs about 11% more per call at 64 B. Also, it costs less per byte above about 16 KiB, where its per-write cost is about 0.65 times that of Preview 1, in real-world scenarios. Because, fuel does not meter host-side work, the WASI per-byte costs are the part of a call's cost that a fuel quota leaves uncovered, and Yu et al. disclose that a guest can employ exactly that gap [22], in actual application; more correctly, both versions rather than should be measured by frameworks that are moving large objects postulate either is faster, to a significant degree.

The growth of about 9 bytes per lifecycle is slight over short runs for sandbox recycling, at least in practice; to be precise, over 10^6 lifecycles it is getting about 9 MB, which a long-lived process should monitor, in real-world scenarios; and so, in our configuration, pooling does not remove the growth; it did not reduce median latency, all things considered.

For calibration, the finding is, in practice, broadly concrete together with negative. In parallel with this, a static or dynamic opcode-class model should not be realized to correct fuel quotas, in the central, at least for now; also, the conducted instruction count is already monitored by the fuel meter, which is routinely why also the remaining difference is not in the class mix, all things considered.

The following protocol gives an operator a basis for quotas on a candidate module before admission, in many respects.

1. Fix the backend and its version. Against this backdrop, along all these dimensions, along all all these dimensions, within this framework, viewed through this lens, within this framework, building on this gather, quotas are, in practice, valid only for that pair, in actual application.
2. Remarkably, build this module at the optimization level the platform will, partially, roll out, and measure fuel for a representative input on that build, at least in practice.
3. Compute the cost of one fuel unit for the module; truly, Measure CPU time for the same input on the same machine class, with enough copies to bound the mean, at least for now.
4. More stringently, set the quota as fuel equal to the target CPU budget divided by an upper bound on the cost per unit, on the whole; to be precise, take the bound as the larger of the upper quartile for the module's kernel class and the upper quartile of the module's own copies, at least empirically. Within this framework, from this viewpoint, a median would generally under-protect code that is extensively slow per operator, in actual application.
5. Beyond this, add the host cost $\alpha + \beta s$ for its representative call count and payload size to the CPU budget, since fuel does not charge for it, if the module makes host calls, in real-world scenarios.
6. Re-measure after any change to the runtime, the backend, the module, or the machine class, on the whole.

Any budget is being given with one stated basis by that strategy, measured on the hardware that will (at least in most settings) run the module; it does not give a sound bound, in actual application; to be precise, operators who are needing a hard bound should measure the worst-case kernel class on their own hardware and treat the finding as a bound only for the measured inputs, to a significant degree.

7 Threats to Validity

Execution isolation and gating. Telemetry gating protocols enforce strict execution isolation before every model; also, thus, also the wall-to-CPU execution ratio serves as a careful runtime verification metric, specifically in our reported dataset, every timed sweep is completely uncontended (median wall-to-CPU ratio 1.000), to a significant degree.

Counters and instrumentation. The dynamic counts come from instrumented copies that return the same values as the originals, proving clear execution, at least for now; the hardware counters are consistently (specifically in this setting) taken as reported by `proc_pid_rusage`, in many respects.

Replicates. 100 models per cell are mostly had by the fast runtimes; in turn, singlepass and WAMR have 10, at least in practice; and thus, also, the WasmEdge interpreter has 3; this reflects the runtime speed profiles, in real-world scenarios.

Fuel meters. Every person person runtime's fuel is its own counter, which is why wasmer's metered fuel is the second operator count, at least in law, in actual application; chiefly, also wasmtime's is its own consumed-fuel count, so the two agree in median, at least in belief, and taken together, despite this, not cell by cell, broadly speaking; truly, the differences are reported as measured, at least empirically. Along these dimensions, within this framework, building on this finding, against this backdrop, winch and Cranelift are disagreeing in 20 cells above noise in this broader context, in actual application in light of this dynamic, at least in practice.

Literature screen. The fifteen spotted literature gaps define the open questions tackled in this examination, and ten keyword searches and six saturation checks were did over one open scholarly index by the screen, restricted to 2024 onward and to peer-reviewed venues, broadly speaking.

Scope of the corpus. The corpus is 50 kernels at the PolyBench MEDIUM dataset; in turn, memory behavior is exercised by the memory kernels and are not WASI system-call sweeps, in actual application; the custom import and not one WASI interface are embraced by this host-call boundary, in actual application. Under these conditions, also, one host, one runtime, and any sink, is instantiated in this principal by the WASI comparison, on the whole.

Recycling harness. Both the Python and isolated Rust harnesses are, in effect, ordinarily monitoring private anonymous memory, stringently quantifying persistent sandbox lifecycle retention, all things considered.

Background load during the campaign. Table D4 recapitulates the sampled 1-minute load (Appendix D). Under these conditions, its median is 4.35, more than half of the samples exceed the admission gate of 4, in many respects; also the maximum is 55.95, broadly speaking; at least in belief, at least for now. Also, thus, the gate is applying to the start of every person person person person person model; and so, at that moment, the per-replicate log does not record the load, yet the log can't share that each copy began below this gate, on the whole. Building on this observation, one more second emulate

that ran under the contention would have, broadly speaking, longer wall time, in actual application; chiefly, also the wall-to-CPU check is the test of time-sharing, within any model, not of co-running load between duplicates, to one significant degree, in real-world scenarios; to be precise, on the gate, absolute times are therefore conditional, in many respects. Under these conditions, from this standpoint, the per-replicate wall-to-CPU ratio, with this median of 1.000 and 95th percentiles of at most 1.035 across the timed configurations (Table 2), and the clear effective clock of 3.65 to 3.73 GHz (Table 5), are disclosing that the timed replicates were not time-shared within the their own thread and ran at any stable clock, all things considered. Under these conditions, against this backdrop, they do not point to that no other process was active between models, and also, that catch is carried by the runtime ratios, on the whole.

Interval assumptions in recycling. The Rust allocator intervals rest on five independent processes per allocator and employ the spread of repeat slopes rather, in actual application; the single-run intervals in Table 8 apply ordinary least-squares standard mismatches, which suppose independent residuals, and memory series from one process are autocorrelated, so all all all those intervals are optimistic, all things considered.

Two scores, one bound. Table 10 scores calibration as the coefficient of change of time divided by prediction, so viewed through this lens, proposition the additive residual is bounded by 1, in actual application; chiefly, viewed through this lens, the two are separate measures, in actual application; under these conditions, also, the idea is any benchmark for what linear class counts can (to some degree) explain, not one bound on the reported score, on the whole.

8 Conclusion

Across 50 kernels, four optimization levels, and twelve runtime configurations, each configuration is computing the same findings, in this central; chiefly, across the cells on Wasmtime's Cranelift backend, the cost of one fuel unit differs by any second factor of 40.8, in real-world scenarios; to be precise, also the solid spread of 8.9 is sharing that most cells are closer together; in the principal, at least for now. Also, thus, also the same kernel is changing its fuel by up to 6.5 times between frontend builds, on the whole; also fuel depends on this backend, in this central, in the primary; more correctly, taken together, wASI Preview 2 writes cost less per byte than Preview 1 writes, to the significant degree, on the whole. In light of this dynamic, also, about 11% more per call at 64 bytes; more exactly, calibrating cost per fuel unit with opcode-class counts, static or dynamic, does not lower dispersion below the uncalibrated fuel baseline, broadly speaking; also thus, building on this detect, linear-memory probes conformed in all 4,800 trials, at least in practice. Against this backdrop, under these conditions, recycling is increasing runtime-side state by about 9 bytes per lifecycle, autonomous of the sandbox's dirty pages and of the allocation strategy, and one host call costs about 61 times

the further native copy at 64 bytes and reaches parity at 4 MiB, all things considered.

The vital limits of instruction-fuel metering across the modern compiler backends are established by our findings and offer quantitative baselines for fair accounting and sandbox recycling in multi-tenant WebAssembly platforms, in many respects.

Declarations

- **Funding:** This author is regularly declaring that no external funding, grants, or financial support were received during the preparation of this manuscript, in many respects.
- **Competing interests:** The author relevantly interests to share.
- **Ethics approval and consent to participate:** In particular, not applicable; the put forward inquiry is, in practice, embedding computational systems benchmarking and does not entail human participants, human data, or animals, in actual application.
- **Consent for publication:** That is principally to say, the author consents to the publication of this manuscript and all associated tables and figures, broadly speaking.
- **Data availability:** The raw per-replicate timing traces, hardware counter logs, linear-memory boundary probe outcomes, host-call and WASI latency samples, and sandbox recycling checkpoints behind every table and figure are, in this setting, archived in the study repository at https://github.com/RaffeOP/Instruction-Fuel-Drift-Boundary-Crossing-Overhead-and-Memory-Safety-in-WebAssembly-Sandboxes, broadly speaking.
- **Materials availability:** The 200 compiled WebAssembly modules and their disassembled opcode manifests are covered in the replication repository, at least empirically.
- **Code availability:** The Rust, C, JavaScript/N-API, and Python benchmarking employs, dynamic bytecode instrumentation scripts, and figure and table generators are, in practice, available in the study repository at https://github.com/RaffeOP/Instruction-Fuel-Drift-Boundary-Crossing-Overhead-and-Memory-Safety-in-WebAssembly-Sandboxes under the MIT License, in real-world scenarios.
- **Author contribution:** Devarsh Patel conceived the study, designed and realized the multi-runtime benchmarking and bytecode instrumentation uses, ran the experimental campaign, statistically estimated, and wrote and revised the manuscript, in the principal, on the whole.

References

[1] Haas, A., Rossberg, A., Schuff, D.L., Titzer, B.L., Holman, M., Gohman, D., Wagner, L., Zakai, A., Bastien, J.: Bringing the web up to speed with WebAssembly. In: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pp. 185–200 (2017). https://doi.org/10.1145/3062341.3062363

[2] Shillaker, S., Pietzuch, P.: Faasm: Lightweight isolation for efficient stateful serverless computing. In: 2020 USENIX Annual Technical Conference (USENIX ATC 20), pp. 419–433 (2020). https://www.usenix.org/conference/atc20/presentation/shillaker

[3] Jangda, A., Powers, B., Berger, E.D., Guha, A.: Not so fast: Analyzing the performance of WebAssembly vs. native code. In: 2019 USENIX Annual Technical Conference (USENIX ATC 19), pp. 107–120 (2019). https://www.usenix.org/conference/atc19/presentation/jangda

[4] Zhang, Y., Liu, M., Wang, H., Ma, Y., Huang, G., Liu, X.: Research on WebAssembly runtimes: A survey. ACM Transactions on Software Engineering and Methodology **34**(8), 1–47 (2025) https://doi.org/10.1145/3714465

[5] Moron, K., Wallentowitz, S.: Benchmarking WebAssembly for embedded systems. ACM Transactions on Architecture and Code Optimization **22**(3), 1–21 (2025) https://doi.org/10.1145/3736169

[6] Khan, S., Malik, T., Hasanov, K.: A taxonomy and empirical evaluation of WebAssembly runtimes for performance and energy across machine learning and arithmetic workloads. IEEE Access **14**, 146359–146379 (2026) https://doi.org/10.1109/access.2026.3733354

[7] Kakati, S., Brorsson, M.: A cross-architecture evaluation of WebAssembly in the cloud-edge continuum. In: 2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pp. 337–346 (2024). https://doi.org/10.1109/ccgrid59990.2024.00046

[8] Kakati, S., Brorsson, M.: Performance and usability implications of multiplatform and WebAssembly containers. In: Proceedings of the 15th International Conference on Cloud Computing and Services Science (CLOSER), pp. 15–25 (2025). https://doi.org/10.5220/0013203200003950

[9] De Palma, G., Giallorenzo, S., Mauro, J., Trentin, M., Zavattaro, G.: WebAssembly at the edge: Benchmarking a serverless platform for private edge cloud systems. IEEE Internet Computing **28**, 37–44 (2024) https://doi.org/10.1109/mic.2024.3513035

[10] Khelifa, S., Bagaa, M., Messaoud, A.O., Ksentini, A.: Case study of WebAssembly runtimes for AI applications on the edge. In: 2024 Global Information Infrastructure and Networking Symposium (GIIS), pp. 1–6 (2024). https://doi.org/10.1109/giis59465.2024.10449907

[11] Khan, S., Malik, T., Hasanov, K.: Performance evaluation of machine learning applications using WebAssembly across different programming languages. In: Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing (HPDC), pp. 1–5 (2025). https://doi.org/10.1145/3731545.3736817

[12] Golec, M., Walia, G.K., Kumar, M., Cuadrado, F., Gill, S.S., Uhlig, S.:

Cold start latency in serverless computing: A systematic review, taxonomy, and future directions. ACM Computing Surveys **57**, 1–36 (2025) https://doi.org/10.1145/3700875

[13] Moebius, F., Pfandzelter, T., Bermbach, D.: Are unikernels ready for serverless on the edge? In: 2024 IEEE International Conference on Cloud Engineering (IC2E), pp. 133–143 (2024). https://doi.org/10.1109/ic2e61754.2024.00022

[14] Chennubhotla, A., Tewari, S., Somvanshi, H., Chinthana, M.J., Deepti, C.: Moving past Docker for serverless: WebAssembly for Kubernetes clusters at the edge. In: 2025 10th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA), pp. 560–565 (2025). https://doi.org/10.1109/icccbda64898.2025.11030528

[15] Perrone, G., Romano, S.P.: WebAssembly and security: A review. Computer Science Review **56**, 100728 (2025) https://doi.org/10.1016/j.cosrev.2025.100728

[16] Rostamipoor, M., Ghavamnia, S., Polychronakis, M.: LeakLess: Selective data protection against memory leakage attacks for serverless platforms. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2025). https://doi.org/10.14722/ndss.2025.230035

[17] Yu, X., Xiong, T., Tang, W., Jiang, Y., Wei, Z., Wang, H., Chen, C., Wang, Y., Peng, B., Qi, Z.: WaSC: Hardening WebAssembly sandboxes via system interface decoupling. ACM Transactions on Architecture and Code Optimization **23**(1), 1–23 (2026) https://doi.org/10.1145/3795882

[18] Coppolino, L., D'Antonio, S., Mazzeo, G., Nardone, R., Romano, L., Schmitt, M.: WASMBOX: A lightweight Wasm-based runtime for trustworthy multi-tenant embedded systems. IEEE Transactions on Emerging Topics in Computing **13**(2), 467–480 (2025) https://doi.org/10.1109/tetc.2024.3409817

[19] Draissi, O., Cloosters, T., Klein, D., Rodler, M., Musch, M., Johns, M., Davi, L.: Wemby's web: Hunting for memory corruption in WebAssembly. Proceedings of the ACM on Software Engineering (ISSTA) **2**, 1326–1349 (2025) https://doi.org/10.1145/3728937

[20] Wang, W., Ji, H., He, P., Zhang, Y., Wu, Y., Zhang, Y.: WAVEN: WebAssembly memory virtualization for enclaves. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2025). https://doi.org/10.14722/ndss.2025.230746

[21] Fink, M., Stavrakakis, D., Sprokholt, D., Chakraborty, S., Ekberg, J.-E., Bhatotia, P.: Cage: Hardware-accelerated safe WebAssembly. In:

Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO), pp. 538–552 (2025). https://doi.org/10.1145/3696443.3708920

[22] Yu, Z., Zhan, D., Ye, L., Yu, H., Zhang, H., Tian, Z.: Exploring and exploiting the resource isolation attack surface of WebAssembly containers. In: Proceedings of the 34th USENIX Security Symposium (USENIX Security 25), pp. 1111–1128 (2025). https://www.usenix.org/conference/usenixsecurity25/presentation/yu-zhaofeng

[23] Narayan, S., Disselkoen, C., Garfinkel, T., Froyd, N., Rahm, E., Lerner, S., Shacham, H., Stefan, D.: Retrofitting fine grain isolation in the Firefox renderer. In: 29th USENIX Security Symposium (USENIX Security 20), pp. 699–716 (2020). https://www.usenix.org/conference/usenixsecurity20/presentation/narayan

[24] Lehmann, D., Kinder, J., Pradel, M.: Everything old is new again: Binary security of WebAssembly. In: 29th USENIX Security Symposium (USENIX Security 20), pp. 217–234 (2020). https://www.usenix.org/conference/usenixsecurity20/presentation/lehmann

[25] Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., Popa, D.-M.: Firecracker: Lightweight virtualization for serverless applications. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), pp. 419–434 (2020). https://www.usenix.org/conference/nsdi20/presentation/agache

[26] Titzer, B.L., Gilbert, E., Teo, B.W.J., Anand, Y., Takayama, K., Miller, H.: Flexible non-intrusive dynamic instrumentation for WebAssembly. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 398–415 (2024). https://doi.org/10.1145/3620666.3651338

[27] Schwarz, T., Kamm, T., Engelke, A.: TPDE: A fast adaptable compiler back-end framework. In: 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 427–439 (2026). https://doi.org/10.1109/cgo68049.2026.11395208

[28] Jiang, S., Zeng, R., Zhou, Y., Lyu, M.R.: Distinguishability-guided test program generation for WebAssembly runtime performance testing. In: 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 768–779 (2025). https://doi.org/10.1109/saner64311.2025.00078

[29] Zeng, R., Jiang, S., Zhao, W., Zhou, Y.: Debugging performance issues in

WebAssembly runtimes via mutation-based inference. In: Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 3475–3486 (2026). https://doi.org/10.1145/3744916.3773141

[30] Tamura, R., Kotani, D., Shudo, K., Okabe, Y.: Bringing together cross-ISA checkpoint/restoration and AOT compilation of WebAssembly programs. In: Proceedings of the 22nd ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes, pp. 114–124 (2025). https://doi.org/10.1145/3759426.3760985

[31] Mao, C., Su, Y., Shan, S., Li, D.: eWAPA: An eBPF-based WASI performance analysis framework for WebAssembly runtimes. In: 2024 IEEE International Conference on Software Services Engineering (SSE), pp. 323–333 (2024). https://doi.org/10.1109/sse62657.2024.00054

[32] Van Kenhove, M., Seidler, M., Vandenberghe, F., Dujardin, W., Hennen, W., Vogel, A., Sebrechts, M., Goethals, T., De Turck, F., Volckaert, B.: Cyber-physical WebAssembly: Secure hardware interfaces and pluggable drivers. In: NOMS 2025-2025 IEEE Network Operations and Management Symposium, pp. 1–7 (2025). https://doi.org/10.1109/noms57970.2025.11073737

[33] Seidler, M., Krause, A., Ulbrich, P.: Wasm-io: Enabling low-level device interaction in WebAssembly for industry automation. ACM Transactions on Embedded Computing Systems **24**, 1–26 (2025) https://doi.org/10.1145/3760387

[34] Ramesh, A., Huang, T., Titzer, B.L., Rowe, A.: Empowering WebAssembly with thin kernel interfaces. In: Proceedings of the Twentieth European Conference on Computer Systems (EuroSys), pp. 1–20 (2025). https://doi.org/10.1145/3689031.3717470

[35] Jansen, M., Kozub, M., Iosup, A., Bonetta, D.: Memory efficient WebAssembly containers. In: 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pp. 1058–1065 (2025). https://doi.org/10.1109/ipdpsw66978.2025.00166

[36] Antunes, B., Hill, D.R.C.: Reproducibility, replicability and repeatability: A survey of reproducible research with a focus on high performance computing. Computer Science Review **53**, 100655 (2024) https://doi.org/10.1016/j.cosrev.2024.100655

[37] Pearce, O., Becker, G., Brink, S., Hanford, N., Yokelson, D., Knox, A., Rountree, B.: HPC benchmarking: Repeat, replicate, reproduce. In: Proceedings of the 3rd ACM Conference on Reproducibility and Replicability, pp. 85–95 (2025). https://doi.org/10.1145/3736731.3746150

[38] Lema, D.G., Sánchez-González, L., Usamentiaga, R., delaCalle, F.J.: Benchmarking deep learning models for surface defect detection: a reproducible and statistically-rigorous approach. Journal of Intelligent Manufacturing **37**, 3001–3018 (2026) https://doi.org/10.1007/s10845-025-02672-8

[39] Clopper, C.J., Pearson, E.S.: The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika **26**(4), 404–413 (1934) https://doi.org/10.1093/biomet/26.4.404

[40] Welch, B.L.: On the comparison of several mean values: An alternative approach. Biometrika **38**(3/4), 330 (1951) https://doi.org/10.2307/2332579

[41] Hoerl, A.E., Kennard, R.W.: Ridge regression: Biased estimation for nonorthogonal problems. Technometrics **12**(1), 55–67 (1970) https://doi.org/10.1080/00401706.1970.10488634

[42] Davison, A.C., Smith, R.L.: Models for exceedances over high thresholds. Journal of the Royal Statistical Society: Series B **52**(3), 393–425 (1990) https://doi.org/10.1111/j.2517-6161.1990.tb01796.x

[43] Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F.: Producing wrong data without doing anything obviously wrong! In: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 265–276 (2009). https://doi.org/10.1145/1508284.1508275

[44] Georges, A., Buytaert, D., Eeckhout, L.: Statistically rigorous Java performance evaluation. In: Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), pp. 57–76 (2007). https://doi.org/10.1145/1297105.1297033

Table A1 Cost per fuel unit μ_ψ in nanoseconds for every kernel under Wasmtime Cranelift, at each frontend optimization level. Each entry is the mean over the timed replicates of that cell.

Kernel	O0	O2	O3	Oz	Kernel	O0	O2	O3	Oz
adv_branch	0.114	0.086	0.086	0.083	pb_cholesky	0.019	0.028	0.029	0.071
cachethrash	0.030	0.032	0.032	0.045	pb_correlation	0.017	0.040	0.041	0.063
chacha20	0.047	0.028	0.028	0.051	pb_covariance	0.016	0.059	0.058	0.063
collatz	0.049	0.055	0.054	0.051	pb_deriche	0.040	0.083	0.082	0.103
crc32	0.028	0.048	0.048	0.053	pb_doitgen	0.022	0.031	0.029	0.031
divrem	0.025	0.030	0.029	0.034	pb_durbin	0.019	0.029	0.029	0.033
fnv1a	0.039	0.066	0.065	0.055	pb_fdtd_2d	0.015	0.020	0.022	0.063
quicksort	0.076	0.108	0.106	0.106	pb_floyd_warshall	0.016	0.046	0.018	0.057
sha256	0.052	0.024	0.023	0.029	pb_gemm	0.018	0.041	0.041	0.081
strsearch	0.048	0.075	0.074	0.058	pb_gemver	0.019	0.038	0.035	0.035
mem_binary_search	0.127	0.163	0.157	0.135	pb_gesummv	0.023	0.042	0.041	0.057
mem_fill_reduce	0.022	0.022	0.021	0.045	pb_gramschmidt	0.018	0.033	0.033	0.043
mem_grow_monotonic	0.084	0.152	0.133	0.093	pb_heat_3d	0.011	0.022	0.023	0.020
mem_hashtable	0.057	0.103	0.095	0.114	pb_jacobi_1d	0.017	0.026	0.028	0.032
mem_oob_offset_wrap	0.023	0.033	0.032	0.052	pb_jacobi_2d	0.014	0.022	0.024	0.053
mem_pointer_chase	0.153	0.462	0.362	0.268	pb_lu	0.019	0.031	0.032	0.072
mem_stream_copy	0.020	0.027	0.026	0.052	pb_ludcmp	0.021	0.026	0.027	0.074
mem_stride_4k	0.055	0.270	0.252	0.138	pb_mvt	0.032	0.060	0.051	0.078
mem_stride_64	0.044	0.040	0.043	0.043	pb_nussinov	0.015	0.021	0.021	0.048
mem_transpose	0.049	0.173	0.167	0.112	pb_seidel_2d	0.038	0.160	0.163	0.139
pb_2mm	0.029	0.037	0.042	0.078	pb_symm	0.024	0.026	0.030	0.059
pb_3mm	0.024	0.040	0.040	0.083	pb_syr2k	0.016	0.039	0.036	0.052
pb_adi	0.026	0.102	0.103	0.072	pb_syrk	0.016	0.022	0.023	0.070
pb_atax	0.032	0.032	0.034	0.034	pb_trisolv	0.019	0.047	0.048	0.061
pb_bicg	0.022	0.041	0.042	0.027	pb_trmm	0.026	0.035	0.037	0.025

Appendix A Cost per fuel unit for every kernel

Table A1 is giving μ_ψ for all 50 kernels at the four frontend levels, under Wasmtime Cranelift, in real-world scenarios; each person person person person entry is the mean over the timed replicates of one cell, which is generally why across the 200 cells the smallest value is consistently (specifically in this setting) 0.011 ns, for `pb_heat_3d` at `-O0`, in the principal, broadly speaking. In light of this dynamic, building on this note, taken together, also the largest is 0.462 ns, for `mem_pointer_chase` at `-O2`, and also, their ratio is the 40.8× amplification factor of Section 5.1; the group medians over all levels are, in essence, thoroughly 0.033 ns for PolyBench/C, 0.050 ns for the integer and cryptographic kernels, and 0.089 ns for the linear-memory kernels, at least for now. Along the dimensions, along all the dimensions, the extremes quoted in Section 5.1 are mostly kernel medians, across the four levels: 0.021 ns for `pb_heat_3d` and 0.315 ns for `mem_pointer_chase`, at least in belief, at least for now.

Appendix B Time ratios by optimization level

Table B2 is routinely breaking the cross-runtime comparison of Table 5 down by frontend level, which is regularly why pooling the four levels reproduces this statistic in the principal summary, all things considered, and at the same level, each entry is the median over the 50 kernels of the per-kernel ratio of median CPU time to Wasmtime Cranelift, at least in practice.

Table B2 Median per-kernel time ratio relative to Wasmtime Cranelift at each optimization level (median over the 50 kernels of the per-kernel ratio of median CPU time). Pooling the four levels reproduces the statistic in Table 5.

Runtime	`-O0`	`-O2`	`-O3`	`-Oz`
V8 (Node.js)	1.00×	0.77×	0.77×	0.54×
WAMR interpreter	16.76×	9.48×	9.49×	5.88×
WasmEdge AOT	1.13×	0.79×	0.76×	0.50×
WasmEdge interpreter	147.35×	82.63×	83.87×	50.67×
Wasmer Cranelift	0.91×	0.85×	0.84×	0.79×
Wasmer Cranelift, metered	1.24×	0.96×	0.94×	1.00×
Wasmer LLVM	1.93×	0.83×	0.83×	0.56×
Wasmer LLVM, metered	1.88×	0.88×	0.86×	0.80×
Wasmer Singlepass	16.88×	19.59×	19.81×	11.52×
Wasmer Singlepass, metered	17.20×	18.92×	19.15×	11.10×
Wasmtime Cranelift (reference)	1.00×	1.00×	1.00×	1.00×
Wasmtime Winch, metered	4.17×	1.91×	1.93×	1.64×

Table C3 Corpus summary. Each kernel is compiled at four frontend optimization levels, giving 200 modules. Median module size is in kibibytes of `wasm32-wasip1` output.

Group	Kernels	Modules	`-O0` KiB	`-O2` KiB	`-O3` KiB	`-Oz` KiB
PolyBench/C numerical	30	120	34.0	18.7	18.8	18.6
Integer and cryptographic	10	40	0.7	0.9	0.9	0.6
Linear-memory behavior	10	40	0.7	1.0	1.0	0.6

The level effect differs by runtime. In this broader context, the WAMR interpreter falls from 16.8× at `-O0` to 5.9× at `-Oz`, and the WasmEdge interpreter from 147× to 51×, in real-world scenarios; chiefly, both interpreters have their lowest ratios at `-Oz`, at least in law; and thus, Wasmer Singlepass is moving the other way, in actual application. In light of this dynamic, its ratio rises from 16.9× at `-O0` to 19.8× at `-O3`, then falls to 11.5× at `-Oz`, in many respects. The factor is reliably largest at `-O0` and `-Oz`; and thus, for Cranelift, metering inflates the median time ratio by one factor between 1.1 and 1.4, computed as the metered entry divided by the unmetered entry in each column of Table B2, on the whole.

Appendix C Corpus

The corpus has 200 modules, created by gathering each of the 50 kernels at four frontend levels, at least in practice. Table C3 reports their sizes. In light of this dynamic, the kernel sources, the build commands, and the per-module clear with paths and byte counts are in the store (`data/corpus_manifest.csv`), in real-world scenarios; against this backdrop, the PolyBench/C modules are the largest, with any median of 34.0 KiB at `-O0` and about 18.7 KiB at the three raised levels, at least empirically. Along these dimensions, the integer and cryptographic modules and the linear-memory modules are below 1.0 KiB at every level, all things considered.

Table D4 Load telemetry sampled during the campaign. The 1-minute load average is read at a fixed interval. The admission gate for each replicate is a load of at most 4. This table reports the whole sampled series, not the load at each replicate's start, which the per-replicate log does not record.

Quantity	Value
Samples	5,340
Span (hours)	17.7
Median 1-minute load	4.35
95th percentile	17.39
Maximum	55.95
Share of samples above the gate of 4	54.3%

Appendix D Load telemetry

Table D4 recapitulates the load series recorded during the campaign; in turn, the series is covering about 17.7 hours and has 5,340 samples, at least for now; its median is 4.35, and 54.3% of the samples are above the admission gate of 4; this gate is doing to the start of each copy, at least empirically. Under these conditions, within this framework, building on this finding, taken together, along these dimensions, also the log data the load over the whole session, not at each model's start; in many respects; also thus, the series exposes the machine was often busy; in turn, duplicates would have been delayed during all those periods by any gate, at least in belief, at least in practice. In this broader context, still the log does not record which emulates waited, which is why section 7 is assessing what this means for the absolute times, at least empirically.

Appendix E Dynamic operator-class shares

Across the 200 cells, table QLATEXz0005q is giving the median share of each operator class, measured from the instrumented execution counts, to any second significant degree; truly, memory loads and stores medianly are sharing QLATEXz0003q and QLATEXz0004q, at least for now; across the 200 cells, the range of each share is being given by the minimum and maximum columns, on the whole. In the instrumented counts, the combine QLATEXz0007q class is basically zero; in turn, proposition QLATEXz0006q is normally excluding it, so that the class shares are remarkably not collinear, at least in practice, and local and global accesses, integer arithmetic, and constants together executedly operate, with median shares of QLATEXz0000q, QLATEXz0001q, and QLATEXz0002q, on the whole.

Appendix F Reproduction map

The store is regularly organized so that each table and figure has one individual making path from raw data.

Table E5 Dynamic operator-class shares across the 200 cells, in percent of executed operations. Shares are computed from the instrumented counts in `dynamic_opcode_mix.csv`.

Operator class	Median (%)	Minimum (%)	Maximum (%)
`int_arith`	17.67	10.04	43.15
`float_arith`	3.49	0.00	20.12
`compare`	2.69	0.00	11.57
`memory_load`	9.89	0.00	26.20
`memory_store`	3.80	0.00	10.00
`branch`	2.94	0.34	11.77
`control`	0.04	0.00	8.33
`call`	0.00	0.00	1.46
`local_global`	35.77	17.35	51.54
`const`	14.60	6.99	34.26
`conversion`	0.03	0.00	11.42
`memory_ops`	0.00	0.00	0.01
`other`	0.00	0.00	5.14

- `src/build_corpus.py` gathers the 50 kernels at four levels and writes `data/corpus_manifest.csv`.
- `scripts/run_full_campaign.sh`, `scripts/run_timing_sweeps.sh`, and `scripts/run_wasi_bench.sh` are, in this setting, running the timed campaign for each runtime, the boundary and host-call sweeps, and the WASI comparison, at least for now.
- `src/rq1_analysis.py`, the cell summaries and the RQ2 to RQ4 measures from the raw rows are computed by `src/memory_boundary_auditor.py`, `src/host_call_benchmarker.py`, `src/recycling_auditor.py`, and `src/pooled_recycling_auditor.py`, at least empirically.
- `src/dynamic_opcode_counter.py` and `src/loop_weighted_calibration.py` are, in effect, framing the opcode-class counts and the RQ5 calibration results, in real-world scenarios.
- `src/make_figures.py` and the shared figures and tables are, in practice, regenerated by `src/make_tables.py`, spanning Figures 1–4 and Tables 2–10, at least empirically.
- `src/make_springer_tables.py`, `src/make_springer_rq_tables.py`, `src/make_springer_appendix.py`, and `src/make_springer_figures.py` regenerate the tables of this Springer version and Figure 6 from the same data, in real-world scenarios.

Appendix G Measurements for every cell

Table G6 lists the measured values for all 200 cells under Wasmtime Cranelift; and thus, along these dimensions, the table reports the did fuel, the median CPU time, the cost per fuel unit; this cycles per fuel unit, with the optimization level in parentheses after each kernel name, in many respects. In light of this dynamic, taken together, building on this finding, within this framework, winch's fuel with Cranelift's for The last column is, put simply, inspecting the same module, to one significant degree; at least in belief; and thus, also thus, winch is agreeing with Cranelift exactly in most cells, at least in practice. Taken together, its differences are stronged in the `-O2` and `-O3` builds of the

ten kernels named in Section 5.1, all things considered; the eight PolyBench kernels in that set differ by between 0.04% and 0.20%; in real-world scenarios; and thus, simultaneously, the two linear-memory kernels differ by 10.9% and 16.7%, in actual application.

Table G6: Every measured cell under Wasmtime Cranelift, with the Winch fuel difference. Fuel is in millions of units, CPU time is the per-cell median of thread CPU time in milliseconds, μ_ψ is the cost per fuel unit in nanoseconds, and the last column gives $(F_{\text{Winch}} - F_{\text{Cranelift}})/F_{\text{Cranelift}}$.

Kernel (level)	Fuel (M)	CPU (ms)	μ_ψ (ns)	Cycles/fuel	Winch diff.
`adv_branch` (O0)	165.0	18.76	0.114	0.43	+0.00%
`adv_branch` (O2)	109.5	9.48	0.086	0.32	+0.00%
`adv_branch` (O3)	109.5	9.42	0.086	0.32	+0.00%
`adv_branch` (Oz)	115.5	9.59	0.083	0.30	+0.00%
`cachethrash` (O0)	276.8	8.41	0.030	0.11	+0.00%
`cachethrash` (O2)	309.1	9.85	0.032	0.12	+0.00%
`cachethrash` (O3)	171.7	5.42	0.032	0.12	+0.00%
`cachethrash` (Oz)	188.7	8.41	0.045	0.16	+0.00%
`chacha20` (O0)	602.7	28.24	0.047	0.17	+0.00%
`chacha20` (O2)	561.4	15.64	0.028	0.10	+0.00%
`chacha20` (O3)	374.2	10.41	0.028	0.10	+0.00%
`chacha20` (Oz)	231.6	11.78	0.051	0.18	+0.00%
`collatz` (O0)	749.4	36.24	0.049	0.18	+0.00%
`collatz` (O2)	486.1	26.80	0.055	0.20	+0.00%
`collatz` (O3)	486.1	26.11	0.054	0.20	+0.00%
`collatz` (Oz)	532.4	26.87	0.051	0.18	+0.00%
`crc32` (O0)	370.1	10.40	0.028	0.10	+0.00%
`crc32` (O2)	206.6	9.95	0.048	0.18	+0.00%
`crc32` (O3)	206.6	9.78	0.048	0.18	+0.00%
`crc32` (Oz)	197.1	10.42	0.053	0.19	+0.00%
`divrem` (O0)	529.2	12.82	0.025	0.09	+0.00%
`divrem` (O2)	384.0	11.44	0.030	0.11	+0.00%
`divrem` (O3)	132.0	3.88	0.029	0.11	+0.00%
`divrem` (Oz)	460.8	15.87	0.034	0.13	+0.00%
`fnv1a` (O0)	268.4	11.53	0.039	0.14	+0.00%
`fnv1a` (O2)	136.8	9.03	0.066	0.24	+0.00%
`fnv1a` (O3)	182.5	11.78	0.065	0.24	+0.00%
`fnv1a` (Oz)	293.6	16.17	0.055	0.20	+0.00%
`mem_binary_search` (O0)	278.7	34.14	0.127	0.46	+0.00%
`mem_binary_search` (O2)	146.0	23.24	0.163	0.58	+0.00%
`mem_binary_search` (O3)	146.0	22.84	0.157	0.59	+0.00%
`mem_binary_search` (Oz)	166.3	21.81	0.135	0.49	+0.00%
`mem_fill_reduce` (O0)	843.1	17.86	0.022	0.08	+0.00%
`mem_fill_reduce` (O2)	780.1	17.16	0.022	0.08	+0.00%
`mem_fill_reduce` (O3)	585.1	12.42	0.021	0.08	+0.00%
`mem_fill_reduce` (Oz)	388.0	17.34	0.045	0.16	+0.00%
`mem_grow_monotonic` (O0)	12.3	1.02	0.084	0.31	+0.00%
`mem_grow_monotonic` (O2)	5.9	0.89	0.152	0.55	+0.00%
`mem_grow_monotonic` (O3)	4.4	0.58	0.133	0.49	+0.00%
`mem_grow_monotonic` (Oz)	4.6	0.43	0.093	0.34	+0.00%
`mem_hashtable` (O0)	96.9	5.47	0.057	0.21	+0.00%
`mem_hashtable` (O2)	115.6	11.73	0.103	0.37	-10.89%
`mem_hashtable` (O3)	115.6	11.03	0.095	0.36	-10.89%

Continued on next page

Kernel (level)	Fuel (M)	CPU (ms)	μ_ψ (ns)	Cycles/fuel	Winch diff.
`mem_hashtable` (Oz)	135.3	15.35	0.114	0.41	+0.00%
`mem_oob_offset_wrap` (O0)	109.6	2.48	0.023	0.08	+0.00%
`mem_oob_offset_wrap` (O2)	25.5	0.83	0.033	0.12	+0.00%
`mem_oob_offset_wrap` (O3)	36.9	1.17	0.032	0.12	+0.00%
`mem_oob_offset_wrap` (Oz)	61.8	3.19	0.052	0.19	+0.00%
`mem_pointer_chase` (O0)	69.2	9.44	0.153	0.54	+0.00%
`mem_pointer_chase` (O2)	21.8	9.63	0.462	1.67	+0.00%
`mem_pointer_chase` (O3)	21.8	7.84	0.362	1.36	+0.00%
`mem_pointer_chase` (Oz)	35.7	9.25	0.268	0.97	+0.00%
`mem_stream_copy` (O0)	220.3	4.21	0.020	0.07	+0.00%
`mem_stream_copy` (O2)	604.2	16.07	0.027	0.10	-16.66%
`mem_stream_copy` (O3)	327.3	8.27	0.026	0.10	-16.66%
`mem_stream_copy` (Oz)	352.5	18.23	0.052	0.19	+0.00%
`mem_stride_4k` (O0)	318.8	16.90	0.055	0.19	+0.00%
`mem_stride_4k` (O2)	104.9	28.09	0.270	0.98	+0.00%
`mem_stride_4k` (O3)	104.9	26.30	0.252	0.93	+0.00%
`mem_stride_4k` (Oz)	172.0	23.90	0.138	0.50	+0.00%
`mem_stride_64` (O0)	314.6	13.94	0.044	0.15	+0.00%
`mem_stride_64` (O2)	110.1	4.25	0.040	0.14	+0.00%
`mem_stride_64` (O3)	220.2	8.89	0.043	0.15	+0.00%
`mem_stride_64` (Oz)	180.4	7.72	0.043	0.16	+0.00%
`mem_transpose` (O0)	283.1	13.65	0.049	0.18	+0.00%
`mem_transpose` (O2)	43.3	7.50	0.173	0.63	+0.00%
`mem_transpose` (O3)	65.0	10.85	0.167	0.62	+0.00%
`mem_transpose` (Oz)	120.2	13.49	0.112	0.41	+0.00%
`pb_2mm` (O0)	988.3	29.25	0.029	0.11	+0.00%
`pb_2mm` (O2)	305.2	11.22	0.037	0.14	+0.00%
`pb_2mm` (O3)	305.2	12.03	0.042	0.15	+0.00%
`pb_2mm` (Oz)	388.4	30.74	0.078	0.29	+0.00%
`pb_3mm` (O0)	1,494.8	32.21	0.024	0.08	+0.00%
`pb_3mm` (O2)	473.3	17.53	0.040	0.14	+0.00%
`pb_3mm` (O3)	473.3	18.09	0.040	0.14	+0.00%
`pb_3mm` (Oz)	577.6	49.17	0.083	0.31	+0.00%
`pb_adi` (O0)	2,185.4	56.24	0.026	0.09	+0.00%
`pb_adi` (O2)	637.9	65.09	0.102	0.38	+0.00%
`pb_adi` (O3)	637.9	65.85	0.103	0.38	+0.00%
`pb_adi` (Oz)	756.0	54.05	0.072	0.27	+0.00%
`pb_atax` (O0)	461.6	14.26	0.032	0.11	+0.00%
`pb_atax` (O2)	276.1	8.84	0.032	0.12	-0.04%
`pb_atax` (O3)	347.3	11.64	0.034	0.12	-0.04%
`pb_atax` (Oz)	335.1	11.40	0.034	0.13	+0.00%
`pb_bicg` (O0)	738.9	15.23	0.022	0.08	+0.00%
`pb_bicg` (O2)	187.3	7.42	0.041	0.15	-0.04%
`pb_bicg` (O3)	178.7	7.36	0.042	0.15	-0.04%
`pb_bicg` (Oz)	189.2	5.16	0.027	0.10	+0.00%
`pb_cholesky` (O0)	4,891.7	91.26	0.019	0.07	+0.00%
`pb_cholesky` (O2)	1,564.7	43.40	0.028	0.10	-0.12%
`pb_cholesky` (O3)	1,564.7	44.69	0.029	0.10	-0.12%
`pb_cholesky` (Oz)	1,957.9	150.08	0.071	0.26	+0.00%
`pb_correlation` (O0)	501.8	8.33	0.017	0.06	+0.00%
`pb_correlation` (O2)	302.2	12.13	0.040	0.15	+0.00%
`pb_correlation` (O3)	453.3	18.65	0.041	0.15	+0.00%
`pb_correlation` (Oz)	178.6	11.19	0.063	0.24	+0.00%
`pb_covariance` (O0)	499.5	8.04	0.016	0.06	+0.00%

Continued on next page

Kernel (level)	Fuel (M)	CPU (ms)	μ_ψ (ns)	Cycles/fuel	Winch diff.
`pb_covariance` (O2)	285.5	16.63	0.059	0.22	+0.00%
`pb_covariance` (O3)	428.2	24.91	0.058	0.22	+0.00%
`pb_covariance` (Oz)	177.9	11.17	0.063	0.23	+0.00%
`pb_deriche` (O0)	194.7	7.78	0.040	0.15	+0.00%
`pb_deriche` (O2)	180.4	14.90	0.083	0.31	+0.00%
`pb_deriche` (O3)	240.5	19.77	0.082	0.31	+0.00%
`pb_deriche` (Oz)	165.5	17.09	0.103	0.38	+0.00%
`pb_doitgen` (O0)	484.4	10.24	0.022	0.08	+0.00%
`pb_doitgen` (O2)	277.0	8.12	0.031	0.11	+0.00%
`pb_doitgen` (O3)	415.5	11.97	0.029	0.10	+0.00%
`pb_doitgen` (Oz)	590.0	19.26	0.031	0.12	+0.00%
`pb_durbin` (O0)	830.1	15.25	0.019	0.06	+0.00%
`pb_durbin` (O2)	508.4	14.57	0.029	0.10	-15.82%
`pb_durbin` (O3)	403.5	11.48	0.029	0.10	-15.82%
`pb_durbin` (Oz)	404.6	13.24	0.033	0.12	+0.00%
`pb_fdtd_2d` (O0)	1,247.3	18.40	0.015	0.05	+0.00%
`pb_fdtd_2d` (O2)	441.9	8.87	0.020	0.07	+0.00%
`pb_fdtd_2d` (O3)	441.9	9.69	0.022	0.07	+0.00%
`pb_fdtd_2d` (Oz)	495.0	32.41	0.063	0.24	+0.00%
`pb_floyd_warshall` (O0)	11,720.4	191.02	0.016	0.06	+0.00%
`pb_floyd_warshall` (O2)	3,021.0	138.99	0.046	0.17	+0.00%
`pb_floyd_warshall` (O3)	3,021.0	52.71	0.018	0.07	+0.00%
`pb_floyd_warshall` (Oz)	3,650.0	219.41	0.057	0.21	+0.00%
`pb_gemm` (O0)	727.8	12.76	0.018	0.07	+0.00%
`pb_gemm` (O2)	677.3	26.70	0.041	0.15	+0.00%
`pb_gemm` (O3)	677.3	26.58	0.041	0.15	+0.00%
`pb_gemm` (Oz)	560.4	46.21	0.081	0.30	+0.00%
`pb_gemver` (O0)	352.5	6.68	0.019	0.07	+0.00%
`pb_gemver` (O2)	302.3	10.01	0.038	0.13	+0.00%
`pb_gemver` (O3)	359.8	11.52	0.035	0.12	+0.00%
`pb_gemver` (Oz)	333.0	10.07	0.035	0.13	+0.00%
`pb_gesummv` (O0)	262.5	6.05	0.023	0.08	+0.00%
`pb_gesummv` (O2)	180.4	7.35	0.042	0.15	+0.00%
`pb_gesummv` (O3)	189.2	7.78	0.041	0.15	+0.00%
`pb_gesummv` (Oz)	235.4	13.45	0.057	0.21	+0.00%
`pb_gramschmidt` (O0)	826.2	14.71	0.018	0.06	+0.00%
`pb_gramschmidt` (O2)	466.7	15.48	0.033	0.12	-0.20%
`pb_gramschmidt` (O3)	233.4	7.54	0.033	0.12	-0.20%
`pb_gramschmidt` (Oz)	274.9	9.92	0.043	0.16	+0.00%
`pb_heat_3d` (O0)	2,944.4	33.11	0.011	0.04	-0.00%
`pb_heat_3d` (O2)	709.0	15.32	0.022	0.08	-0.00%
`pb_heat_3d` (O3)	709.0	16.51	0.023	0.08	-0.00%
`pb_heat_3d` (Oz)	733.1	14.53	0.020	0.07	-0.00%
`pb_jacobi_1d` (O0)	369.9	6.44	0.017	0.05	+0.00%
`pb_jacobi_1d` (O2)	355.6	9.09	0.026	0.09	+0.00%
`pb_jacobi_1d` (O3)	559.9	15.16	0.028	0.10	+0.00%
`pb_jacobi_1d` (Oz)	420.8	13.27	0.032	0.12	+0.00%
`pb_jacobi_2d` (O0)	1,384.0	19.36	0.014	0.05	+0.00%
`pb_jacobi_2d` (O2)	495.2	10.78	0.022	0.08	+0.00%
`pb_jacobi_2d` (O3)	495.2	11.85	0.024	0.08	+0.00%
`pb_jacobi_2d` (Oz)	532.6	28.76	0.053	0.20	+0.00%
`pb_lu` (O0)	5,592.6	99.76	0.019	0.07	+0.00%
`pb_lu` (O2)	1,774.3	54.47	0.031	0.11	-0.11%
`pb_lu` (O3)	1,774.3	56.99	0.032	0.11	-0.11%

Continued on next page

Kernel (level)	Fuel (M)	CPU (ms)	μ_ψ (ns)	Cycles/fuel	Winch diff.
`pb_lu` (Oz)	2,224.9	175.95	0.072	0.27	+0.00%
`pb_ludcmp` (O0)	5,327.7	108.84	0.021	0.07	+0.00%
`pb_ludcmp` (O2)	1,713.9	43.26	0.026	0.09	-0.11%
`pb_ludcmp` (O3)	1,713.6	46.28	0.027	0.09	-0.11%
`pb_ludcmp` (Oz)	2,186.6	175.56	0.074	0.28	+0.00%
`pb_mvt` (O0)	278.9	8.89	0.032	0.12	+0.00%
`pb_mvt` (O2)	285.8	15.79	0.060	0.21	+0.00%
`pb_mvt` (O3)	339.4	16.58	0.051	0.19	+0.00%
`pb_mvt` (Oz)	410.9	31.74	0.078	0.29	+0.00%
`pb_nussinov` (O0)	1,993.3	29.86	0.015	0.06	+0.00%
`pb_nussinov` (O2)	551.0	11.67	0.021	0.08	-0.18%
`pb_nussinov` (O3)	551.0	11.73	0.021	0.08	-0.18%
`pb_nussinov` (Oz)	656.7	31.32	0.048	0.17	+0.00%
`pb_seidel_2d` (O0)	2,923.3	109.60	0.038	0.14	+0.00%
`pb_seidel_2d` (O2)	844.0	134.59	0.160	0.60	+0.00%
`pb_seidel_2d` (O3)	844.0	137.38	0.163	0.60	+0.00%
`pb_seidel_2d` (Oz)	955.5	134.50	0.139	0.52	+0.00%
`pb_symm` (O0)	506.7	11.45	0.024	0.09	+0.00%
`pb_symm` (O2)	545.3	14.14	0.026	0.09	+0.00%
`pb_symm` (O3)	544.4	14.98	0.030	0.10	+0.00%
`pb_symm` (Oz)	393.3	25.55	0.059	0.22	+0.00%
`pb_syr2k` (O0)	593.0	9.44	0.016	0.06	+0.00%
`pb_syr2k` (O2)	224.1	8.28	0.039	0.13	+0.00%
`pb_syr2k` (O3)	224.1	7.94	0.036	0.13	+0.00%
`pb_syr2k` (Oz)	711.3	37.27	0.052	0.19	+0.00%
`pb_syrk` (O0)	801.3	13.09	0.016	0.06	+0.00%
`pb_syrk` (O2)	655.3	14.02	0.022	0.08	+0.00%
`pb_syrk` (O3)	655.3	15.10	0.023	0.08	+0.00%
`pb_syrk` (Oz)	832.0	60.15	0.070	0.26	+0.00%
`pb_trisolv` (O0)	309.8	5.84	0.019	0.06	+0.00%
`pb_trisolv` (O2)	430.5	20.09	0.047	0.17	+0.00%
`pb_trisolv` (O3)	430.5	20.46	0.048	0.17	+0.00%
`pb_trisolv` (Oz)	381.9	23.28	0.061	0.23	+0.00%
`pb_trmm` (O0)	316.6	5.52	0.026	0.09	+0.00%
`pb_trmm` (O2)	531.6	17.74	0.035	0.13	+0.00%
`pb_trmm` (O3)	530.6	18.29	0.037	0.13	+0.00%
`pb_trmm` (Oz)	127.5	3.24	0.025	0.09	+0.00%
`quicksort` (O0)	184.9	14.08	0.076	0.28	+0.00%
`quicksort` (O2)	116.7	12.60	0.108	0.40	+0.00%
`quicksort` (O3)	116.7	12.32	0.106	0.39	+0.00%
`quicksort` (Oz)	119.7	12.67	0.106	0.39	+0.00%
`sha256` (O0)	485.6	23.59	0.052	0.19	+0.00%
`sha256` (O2)	444.4	10.83	0.024	0.09	+0.00%
`sha256` (O3)	578.8	13.26	0.023	0.09	+0.00%
`sha256` (Oz)	530.0	14.94	0.029	0.10	+0.00%
`strsearch` (O0)	228.6	10.42	0.048	0.17	+0.00%
`strsearch` (O2)	125.8	9.41	0.075	0.27	+0.00%
`strsearch` (O3)	209.6	15.35	0.074	0.27	+0.00%
`strsearch` (Oz)	227.4	13.32	0.058	0.21	+0.00%

Appendix H Linear-memory probe outcomes

Table H7 is counting for each of the six guard configurations and each of the eight probes, at least in principle, at least in law. In this broader context,

against this backdrop, 100 trials are, in practice, covered by each person row in light of this dynamic; truly, within this framework, 3,000 stated returns and 1,800 graceful traps were created by the campaign, on the whole. Within this framework, viewed through this lens, taken together, still also, no probe ended in any further silent corruption or the host crash, at least in law; and thus, the counts in each row sum to the number of trials, so the conformance intervals in Section 5.2 rest on the full set of 4,800 trials, at least empirically.

Table H7: Linear-memory probe outcomes for every configuration and probe. Each probe row covers the 100 trials of one probe under one guard configuration. A specified return and a graceful trap both conform to the specification.

Probe	Trials	Graceful	Spec. return
`default_2gib_guard`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`no_growth_reservation`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`reduced_64kib_guard`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`zero_guard`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`zero_guard_explicit_checks`			

Continued on next page

Table I8 Host-call and WASI costs by payload size. Host-call entries are medians of the per-call time, in microseconds, for the wasm boundary and for the native copy-and-validate. WASI entries are the median over replicates of the full-minus-baseline CPU time per write, in microseconds, for Preview 1 and Preview 2. The last column is the Preview 2 to Preview 1 ratio.

Payload	Host call (µs)	Native (µs)	Ratio	P1 write (µs)	P2 write (µs)	P2/P1
64 B	5.628	0.092	61.2	0.801	0.886	1.11
1 KiB	5.814	0.220	26.4	0.966	1.049	1.09
16 KiB	7.022	1.134	6.2	4.838	3.313	0.68
256 KiB	38.541	25.848	1.5	72.999	45.337	0.62
1 MiB	96.220	76.961	1.3	290.427	184.654	0.64
4 MiB	542.554	549.053	1.0	1161.295	742.152	0.64
16 MiB	2,273.106	1,448.430	1.6	4655.712	3006.383	0.65

Probe	Trials	Graceful	Spec. return
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`zero_guard_no_growth_reservation`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0

Appendix I Host-call and WASI sweeps

Table I8 is giving the medians behind Figure 5 and the WASI comparison of Section 5.3; to be precise, the host-call ratio falls from 61 at 64 B to 1.0 at 4 MiB, at least in practice; at 16 MiB it rises again to 1.6, in the primary, at least empirically. Also the sweep does not isolate the cause of that rise; the table reports medians over models, so the dispersion is not uncovered here, all things considered; all things considered. Also, thus, also the replicate-level data are in `data/rq3_wasi_p1_p2.csv`, all things considered; truly, within this framework, building on this gather, the Preview 2 to Preview 1 ratio is above 1 at 64 B. Also, falls below 0.7 from 16 KiB upward, in the principal, all things considered. Along these dimensions, still the merit of Preview 2 for large writes holds across the measured range, at least for now; more exactly, the WASI per-write cost is, in effect, stated as the replicate-wise full-minus-baseline CPU time divided by the number of writes, in many respects.

Appendix J Per-kernel Welch ANOVA

Table J9 reports the Welch ANOVA for each of the 50 kernels, across the four frontend levels, in real-world scenarios; chiefly, the median partial η^2 is 0.956 for the integer kernels, 0.947 for the linear-memory kernels, and 0.926 for PolyBench, with minima of 0.726, 0.309, and 0.629, the pattern encapsulated in Table 4, in many respects. In light of this dynamic, taken together, from this outlook, taken together, every kernel greatly is effectuating, to a significant degree.

Table J9: Welch's heteroscedastic ANOVA of log-transformed CPU time across the four frontend levels, for each kernel under Wasmtime Cranelift. Degrees of freedom between groups are 3 for every kernel. The within-group degrees of freedom are fractional because of the Welch correction.

Kernel	Group	F_W	df	p	η_p^2
`adv_branch`	integer	65,644.5	216.1	$< 10^{-300}$	0.998
`cachethrash`	integer	124,317.5	187.5	6.31×10^{-309}	0.994
`chacha20`	integer	58,066.1	203.5	3.93×10^{-298}	0.996
`collatz`	integer	5,369.3	207.9	1.20×10^{-196}	0.986
`crc32`	integer	420.1	216.8	5.01×10^{-90}	0.726
`divrem`	integer	441,886.3	209.9	$< 10^{-300}$	0.996
`fnv1a`	integer	56,967.3	201.1	2.57×10^{-294}	0.873
`mem_binary_search`	memory	433.9	170.6	1.38×10^{-79}	0.792
`mem_fill_reduce`	memory	5,109.2	211.3	7.59×10^{-197}	0.939
`mem_grow_monotonic`	memory	3,253.4	210.6	4.69×10^{-176}	0.956
`mem_hashtable`	memory	8,997.3	209.9	2.35×10^{-221}	0.986
`mem_oob_offset_wrap`	memory	154,549.4	199.4	$< 10^{-300}$	0.998
`mem_pointer_chase`	memory	297.1	170.3	2.07×10^{-67}	0.309
`mem_stream_copy`	memory	15,084.9	170.1	3.94×10^{-206}	0.988
`mem_stride_4k`	memory	447.1	187.7	3.24×10^{-85}	0.823
`mem_stride_64`	memory	2,280.2	212.7	2.16×10^{-161}	0.911
`mem_transpose`	memory	6,567.3	215.6	1.44×10^{-211}	0.983
`pb_2mm`	polybench	6,353.6	189.4	9.46×10^{-190}	0.946
`pb_3mm`	polybench	2,638.3	202.9	3.44×10^{-162}	0.883
`pb_adi`	polybench	410.7	201.1	1.80×10^{-85}	0.759
`pb_atax`	polybench	2,076.0	187.1	2.54×10^{-143}	0.852
`pb_bicg`	polybench	3,421.9	184.0	4.10×10^{-161}	0.958
`pb_cholesky`	polybench	2,385.6	199.2	8.80×10^{-156}	0.889
`pb_correlation`	polybench	1,645.1	176.2	1.46×10^{-128}	0.927
`pb_covariance`	polybench	84,003.0	203.8	$< 10^{-300}$	0.994
`pb_deriche`	polybench	27,433.4	200.3	1.60×10^{-261}	0.991
`pb_doitgen`	polybench	1,102.6	182.9	8.28×10^{-117}	0.867
`pb_durbin`	polybench	2,732.0	197.6	1.57×10^{-160}	0.881
`pb_fdtd_2d`	polybench	10,773.1	206.8	8.03×10^{-227}	0.986
`pb_floyd_warshall`	polybench	78,419.6	209.3	$< 10^{-300}$	0.913
`pb_gemm`	polybench	7,044.5	206.4	1.53×10^{-207}	0.964

Continued on next page

Kernel	Group	F_W	df	p	η_p^2
`pb_gemver`	polybench	775.4	170.8	3.37×10^{-99}	0.629
`pb_gesummv`	polybench	22,101.6	192.6	4.64×10^{-244}	0.964
`pb_gramschmidt`	polybench	1,461.2	210.6	1.28×10^{-140}	0.826
`pb_heat_3d`	polybench	16,816.0	206.7	1.27×10^{-246}	0.993
`pb_jacobi_1d`	polybench	10,345.6	172.7	1.37×10^{-194}	0.972
`pb_jacobi_2d`	polybench	4,472.9	207.5	3.22×10^{-188}	0.974
`pb_lu`	polybench	1,551.6	188.0	2.52×10^{-132}	0.913
`pb_ludcmp`	polybench	2,636.7	181.0	4.57×10^{-149}	0.942
`pb_mvt`	polybench	5,530.6	176.8	1.79×10^{-174}	0.921
`pb_nussinov`	polybench	96,425.4	206.9	$< 10^{-300}$	0.885
`pb_seidel_2d`	polybench	1,185.9	197.2	7.30×10^{-126}	0.926
`pb_symm`	polybench	223.9	194.6	7.99×10^{-63}	0.709
`pb_syr2k`	polybench	35,655.4	211.1	3.15×10^{-285}	0.989
`pb_syrk`	polybench	6,786.4	210.0	1.03×10^{-208}	0.989
`pb_trisolv`	polybench	21,282.6	211.3	7.14×10^{-262}	0.997
`pb_trmm`	polybench	12,939.7	168.2	1.19×10^{-198}	0.901
`quicksort`	integer	816.2	207.2	1.93×10^{-114}	0.896
`sha256`	integer	2,633.6	201.4	2.78×10^{-161}	0.926
`strsearch`	integer	26,424.2	211.5	6.24×10^{-272}	0.919

Declarations

Funding. no particular funding for the current study was, in effect, received by The authors, in many respects.

Conflict of interest. The authors are, in practice, declaring that they have no conflict of interest, in the primary, in the primary.

Data availability. All empirical datasets, raw micro-traces, and analysis artifacts created during the present inquiry are publicly accessible in the replication repository at https://github.com/RaffeOP/Instruction-Fuel-Drift-Boundary-Crossing-Overhead-and-Memory-Safety-in-WebAssembly-Sandboxes.

Code availability. All benchmark source code, harness implementations, and reproduction scripts are available in the replication repository at https://github.com/RaffeOP/Instruction-Fuel-Drift-Boundary-Crossing-Overhead-and-Memory-Safety-in-WebAssembly-Sandboxes.

Authors' contributions. Devarsh Patel designed the experimental framework, implemented the measurement harnesses, conducted the empirical benchmark sweeps, and drafted the manuscript. Ranjithkumar S supervised the research, formulated the methodological verification protocols, and revised the manuscript.